

Decompile & Debug

Exploring the relationship between the high level
source code we write and the binary machine
code that powers our computers.

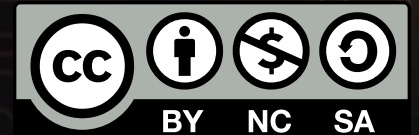


Blah, Blah, Blah

- Presentation by Jim McKeeth
- New & original materials licensed under Creative Commons **BY-NC-SA 4.0**
- Research pulled from experience and various source, including product and project pages, 3rd party publications, and large language models
- Independently verified as possible, considered biased opinion otherwise

- Icons: Icons 8 | Noun Project | Font Awesome | Google
- Graphics: Pixabay | Pexels | Unsplash
- Images & reference: Wikipedia | Wikimedia
- LLM Research & Infographics: Google Gemini
- Image Editing: Midjourney | banana ovo.re | Gimp
- Slides: Slides.com | Reveal.js

📄 slides.com/jimmckeeth/decompile-debug-bsd
github.com/jimmckeeth/decompile-debug-bsd



Why?



THE LAW OF LEAKY ABSTRACTIONS:

HIGH-LEVEL
ABSTRACTION
(Ideal)

THE LEAK
(Abstraction Falls)

UNDERLYING
COMPLEXITY
SEEPS THROUGH

TO OPERATE
EFFECTIVELY UP HERE,
YOU MUST UNDERSTAND
DOWN THERE.

LOW-LEVEL REALITY
(Understanding Required)

Memory Management, Network
Latency, File System Locks, Driver
Crashes, Hardware Interrupts.

THE LEAK
(Abstraction Falls)

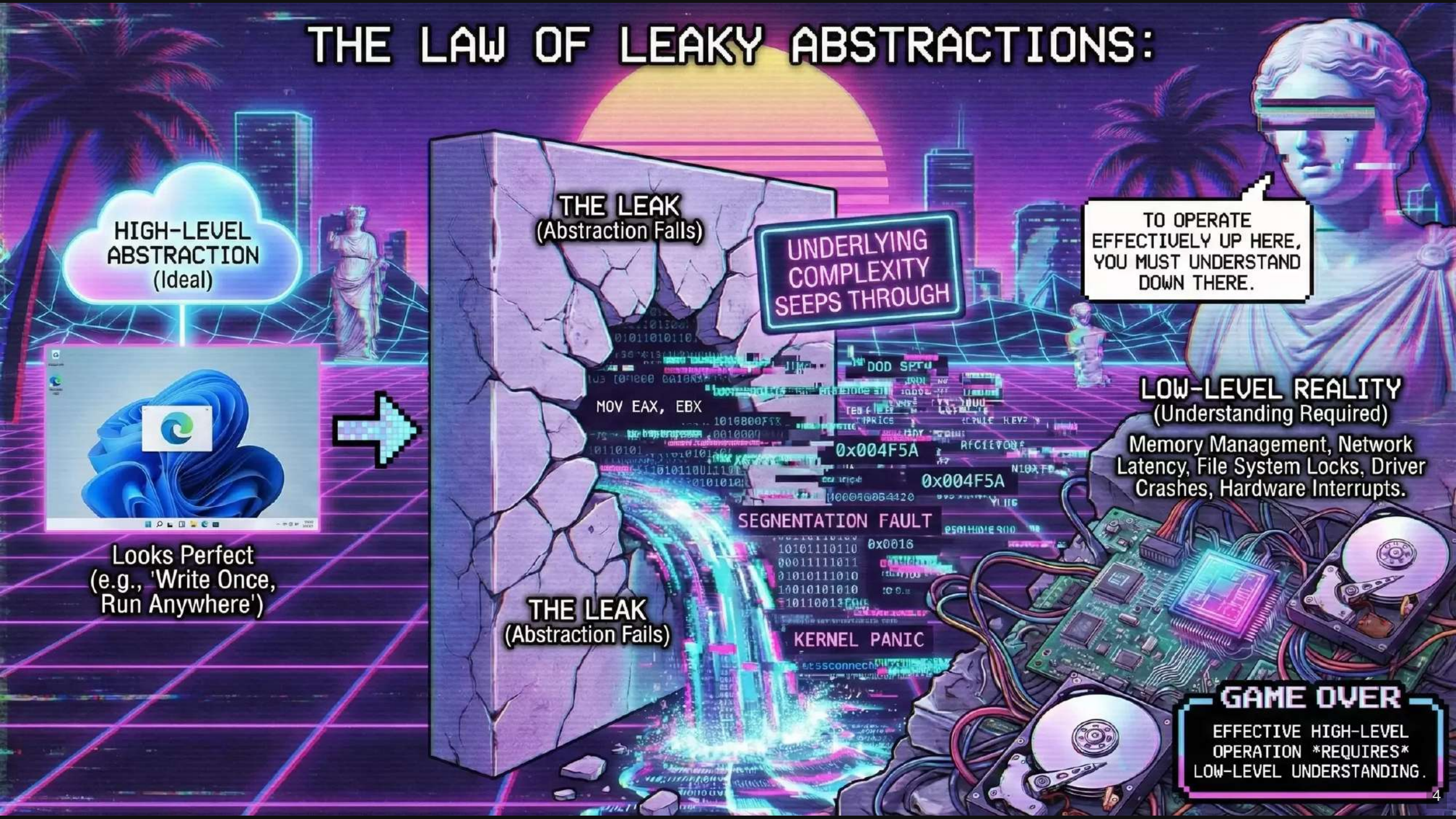
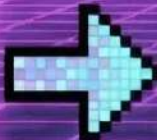
SEGMENTATION FAULT

KERNEL PANIC

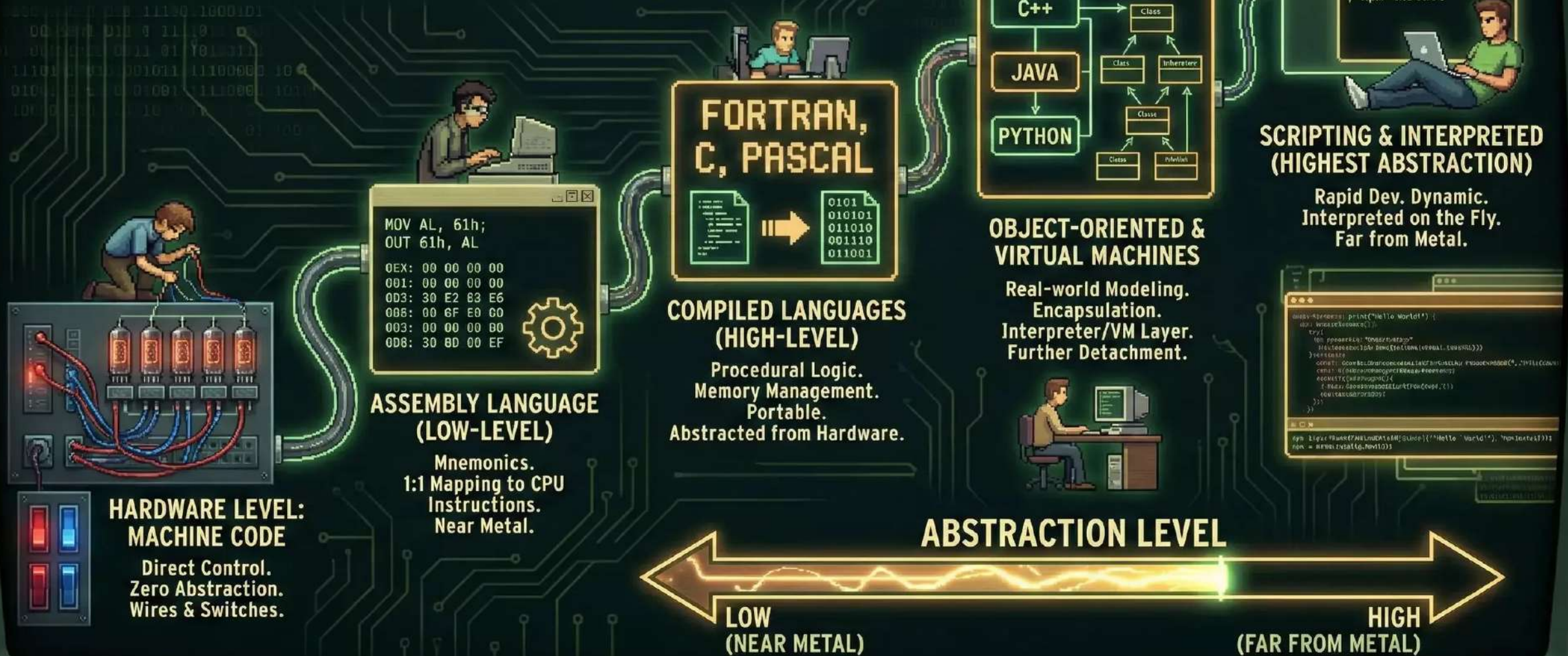
GAME OVER

EFFECTIVE HIGH-LEVEL
OPERATION *REQUIRES*
LOW-LEVEL UNDERSTANDING.

Looks Perfect
(e.g., 'Write Once,
Run Anywhere')



EVOLUTION OF PROGRAMMING: THE ABSTRACTION ASCENSION





```
int main() {  
    print("Hello");  
    int coun= 0;  
    int xo = xa;  
    int num; res ale;  
    int name *as;  
    var loma = assem;  
    var nume semmicn;  
    darmainl = exper;  
    carmpil1 = sri;  
}  
  
int main()  
{  
    print("Hello")  
    int *arax;  
    int = 1;  
    if(!anass)  
        source("%all w  
    return 0;  
}
```



Debug.exe

Originally written by Tim Paterson for 86-DOS

Microsoft hired him & added it to MS-DOS

Still available through Windows XP

Also in FreeDOS, DR-DOS, and other clones.





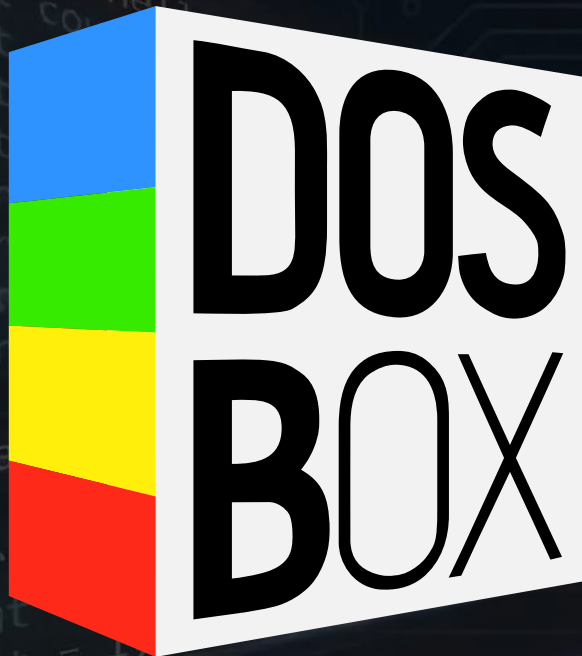
DOSEBOX

C:\>cd DosBox "Way more FPS than Counterstrike!"

- The original - launched in 2002
- Focused on game emulation
- Emulates 286/386/486 real and protected modes
- Also emulates relevant hardware
- The defacto standard for DOS emulation on *many platforms*
- Development slowed in late 2010s
- Last major release 0.74-3 is from 2019
- dosbox.com
- sourceforge.net/projects/dosbox



Alternatives



dosbox-staging.org



dosbox-x.com

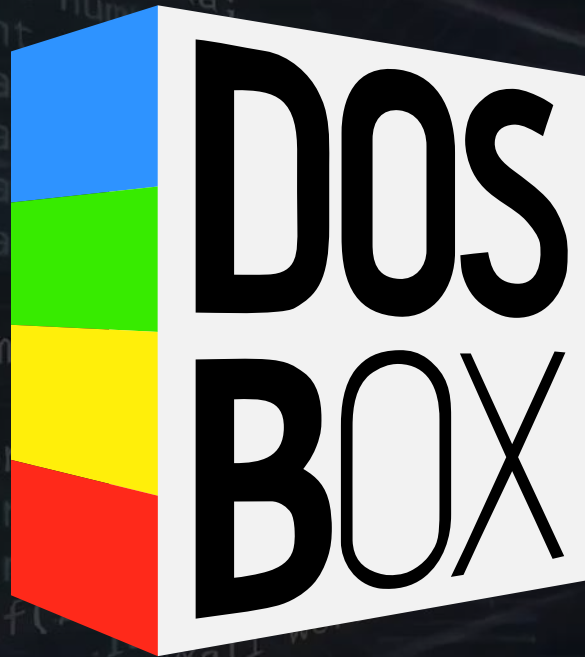


[github.com/schellingb/
dosbox-pure](https://github.com/schellingb/dosbox-pure)

See Also: vdos.info (DOS line of business app focus)

github.com/MartyShepard/DOSBox-Optionals (Win 95 gaming)

DOSBox Staging



dosbox-staging.org

- Latest Release 0.82.2 on Jun 17, 2025
- Focus:
 - High-fidelity Gaming
 - Modern Hardware
 - "Drop-in" Replacement for DosBox
- Dropped support for legacy OS (Windows 7 or older)
- Includes authentic adaptive CRT emulation
- Sensible Defaults

DosBox Pure



- Focused on Simplicity & Ease of Use
- Latest release Pure 1.0 Preview 4 from Nov 2025
- Built for RetroArch/Libretro (game emulator frontends)
- Save states with automatic rewind
- Adds controller support

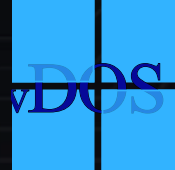
github.com/schellingb/dosbox-pure

DOSBox-X



dosbox-x.com

- Focused on completeness
- A general-purpose x86 emulator that also supports gaming
- Latest Release: 2026.01.02 (Jan 2, 2026)
- GUI and Menu Bar configuration
- Clipboard support
- Good documentation and guides
 - dosbox-x.com/wiki/
 - dosbox-x.com/wiki/Guide:Installing-FreeDOS
 - dosbox-x.com/wiki/Guide:Clipboard-support-in-DOSBox-X
 - dosbox-x.com/wiki/DOSBox-X's-Feature-Highlights



	DOSBox Staging	DOSBox-X	DOSBox Pure	vDos	DOSBox Optionals
Primary Focus	Modern Gaming/Accuracy	Complete System Emulation	Ease of Use/Controller	Business Apps	Win-Integrated Gaming
Codebase	Modern C++ (Clean)	Legacy C++ (Extensive)	Custom / Libretro base	Custom	ECE + Tweaks
Host OS Support	Win/Mac/Linux (Modern)	Win/Mac/Linux/DOS	Win/Mac/Linux	Windows Only	Windows Only
Cycle Accuracy	High (Gaming tuned)	High (Configurable)	Medium	N/A	High
Win 9x Support	Experimental	Excellent (Drivers included)	Automated Install	None	Good
PC-98 Support	No	Yes	No	No	No
Save States	No	Yes (Robust, 100 slots)	Yes (w/ Rewind)	No	Basic
Printing	No	Yes (LPT -> PDF/PNG)	No	Yes (Native)	No
File Locking	Basic (Risk of corruption)	Experimental	Basic	Robust (Win API)	Basic
Input Method	Keyboard/Mouse	Keyboard / Mouse / GUI	Gamepad Mapper/OSK	Keyboard	Keyboard/Mouse
Shaders/CRT	Built-in Dynamic	External Shaders	RetroArch Shaders	None	Integrated

Che Guevara never used Windows.

You: guest [login](#) [register](#)[MS-DOS books](#) — [buy link here](#)**OS****5 recently added:**[DOSVAX PS17 JA](#)[OpenVMS x86-64 V9.2-3 9.2-3](#)[DR DOS 5.0](#)[MS-DOS 5.0 Spanish ACER 5.0 ES](#)[FlexOS 4.01](#)**5 most popular:**[MS DOS 6.00](#)([Sources included](#)) (13575)[MS DOS Boot Disk 7.10](#) (10005)[OS/2 2.1](#) (7961)[MD DOS Boot Disk 7.10](#) (7117)[DOS32 Advanced 7.33](#) (6005)**System****5 recently added:**[FlexOS 286 v1.3 1.3](#)[Doors FR](#)[MS-DOS 3.3 German 3.3 Release](#)[1.2 DE](#)[ThunderBYTE Anti-Virus for](#)[Windows 95 8.10](#)[ReneSis SVG 1.1.1 viewer Plugins 1.1.1](#)**5 most popular:**[Visual Basic 3.0](#) (8637)[Microsoft BASIC Professional](#)[Development System 7.1](#) (6885)[Norton Commander for Windows](#)[2.01](#) (6739)[DOS4GW 2.01](#) (5570)[SoftICE Driver Suite 2.0.1](#) (5188)**DBMS****5 recently added:**[DataEase 4.5 for DOS 4.5](#)[3by5 3.0](#)[dBase IV 4](#)[Visual Objects 1.0a 1.0a](#)[Visual Objects 1.0d Update 1.0d](#)**5 most popular:**[Borland dBASE IV 2.0](#) (7529)[FoxBASE 2.00](#) (4523)[dBASE Compiler 1.0](#) (3271)[CA-Visual Objects for Windows +](#)[upgrade 1.0](#) (2592)[Foxpro for SCO Unix 2.6](#) (2531)**Office****5 recently added:**[pc-write 1.01](#)[Wordtech systems 1.1](#)[Adobe Acrobat Reader 8.0 8.0 PT](#)[Symphony 1.1 ES](#)[WordStar 7 ES](#)**5 most popular:**[WordStar 4.00](#) (10059)[Microsoft Word for Windows](#)[2.0c](#) (5751)[Microsoft Word for Windows](#)[1](#) (4430)[123 4.0](#) (4338)[Assistant Series 2.00](#) (3926)**Utility****5 recently added:**[The - Handy -DOS Utilities](#)[Know Dos Volume 1](#)[Turbo Debugger v5.0 5.0](#)[TURBO BAT 3.10](#)[Palert 2.4](#)**5 most popular:**[Disk Dupe Pro 7.0](#) (4749)[Checkit 3.0](#) (2641)[Advanced CAB Repair 1.0](#) (2205)[ACE, AIN, ARC, ARJ, COM, DWC,](#)[EXE, GZ, LHA, LZH, PAK, RAR,](#)[TAR,](#) (2150)[PC Tools Deluxe 6.0 ES](#) (2133)**Communication****5 recently added:**[View232 1.11](#)[Trillian v3.1.10.0](#)[Antenna Calculation and Design](#)[Tools 1](#)[KIRSCHBAUM_NETZ_153 1.53 DE](#)[DOSCIM 3.1](#)**5 most popular:**[LAN Manager 2.2c](#) (3192)[LapLink Pro 4.00](#) (2427)[LapLink 6](#) (1869)[LANtastic 6.0](#) (1707)[NeoTrace Pro 3.25](#) (1697)**Games****5 recently added:**[THE LAS VEGAS BINGO CLUB Ver 1](#)[Poffessor Erno's Rubik's](#)[MAGIC`CUBE ver 1](#)[3 Ancient games of PIRADA](#)[CHEXO](#)[BASSTOUR 4.2](#)**5 most popular:**[Zak McKracken and the Alien](#)[Mindbenders](#) (8297)[Police Quest](#) (7937)[Civilization](#) (7212)[Maniac Mansion](#) (6194)[Bumpy's Arcade Fantasy](#) (5575)**Multimedia****5 recently added:**[Procyon Pro 2.07 KO](#)[GS1000R Serial MIDI Driver KO](#)[Allegro 1.2](#)[Turbo Trax](#)[dj mix station 3 2.03](#)**5 most popular:**[Clone CD 4.2.0.2](#) (6818)[XingMPEG Player 3.02](#) (3947)[PrintShop Deluxe](#) (2289)[CorelDRAW! 2.0](#) (2026)[MPEGTool 1.04](#) (1916)**Other****5 recently added:**[Bartender 2.0](#)[Uninterrupted Interrupts 1.0](#)[Microsoft Programmer's Library 1.0](#)[1.0](#)[LAN Manager Programmer's Toolkit](#)[\(LMPTK\) 2.1](#)[AutoCAD R14 - Patches and Utilities](#)[N/A](#)**5 most popular:**[Mathcad PLUS 5.0](#) (9841)[AutoCAD R13 R13](#) (8458)[Photoshop 3.4](#) (5627)[3D Home Architect 1.4](#) (4505)[Mathcad 2.5](#) (3372)<https://vetusware.com/>

MS-DOS 6.22

Originally [86-DOS](#), written by Tim Paterson of Seattle Computer Products, DOS was a rough clone of CP/M for 8086 based hardware. Microsoft purchased it and licensed it to IBM for use with Microsoft's IBM PC language products. In 1982, Microsoft began licensing DOS to other OEMs that ported it to their custom x86 hardware and IBM PC clones.

For IBM-specific releases, please see the [IBM PC-DOS product page](#).

Available releases

[1.x](#) [2.x](#) [3.00](#) [3.10](#) [3.20](#) [3.21](#) [3.30](#) [3.31](#) [4.0x](#) [5.0](#) [6.0](#) [6.20](#) [6.21](#)

[6.22](#)

[7.1 \(CDU\)](#)

<https://winworldpc.com/product/ms-dos/622>

Release notes

Microsoft DOS 6.22 was the last standalone version from Microsoft. It was also the last from Microsoft to run on an 8088, 8086, or 286.

6.22 adds DriveSpace, a replacement for DOS 6.20's DoubleSpace drive compression that was removed in 6.21.

There's a really detailed tutorial located at <http://legroom.net/howto/msdos> that gives tips on how to customize DOS. We suggest you follow this tutorials suggestions for setting up and customizing DOS. However, if you're installing to a virtual machine, writing the disk images to actual floppies isn't really necessary.

Downloads

Information

Product type

[OS](#)

Vendor

[Microsoft](#)

Release date

1994

Minimum CPU

8088

Minimum RAM

512KB

Minimum free disk space

5MB

User interface

Text

Platform

[DOS](#)

Download count

1445 (570 for release)

FreeDOS

- An open source DOS-compatible operating system
- Compatible with and includes
 - Classic games
 - Legacy applications
 - Developer tools
- freedos.org - Also links to other resources
- github.com/FDOS/kernel
- gitlab.com/FreeDOS
- gitlab.com/FreeDOS/docs



SECOND EDITION

PC

INTERRUPTS

A PROGRAMMER'S REFERENCE TO
BIOS, DOS, AND THIRD-PARTY CALLS

RALF BROWN & JIM KYLE

Interrupts

- Ralf Brown's Interrupt List (RBIL)
 - 9600 Interrupt entries
 - 5400 tables
- Print version was 1200 pages
- Last revision 61 (17 July 2000)
 - cs.cmu.edu/~ralf/files.html (source)
 - delorie.com/djgpp/doc/rbinter
 - ctyme.com/rbrown.htm
 - fd.lod.bz/rbil



DEBUG.EXE: THE HEX EDITING ARSENAL

- CRACK & PATCH GAMES



QUEST.EXE

C:\>DEBUG QUEST.EXE

LIVES: 3 (0x03)

➡ 4B2C:03

1. THE TARGET (GAME FILE)



2. THE HEX EDITOR (DEBUG)



3. THE PATCHED GAME

REAL vs. PROTECTED MODE

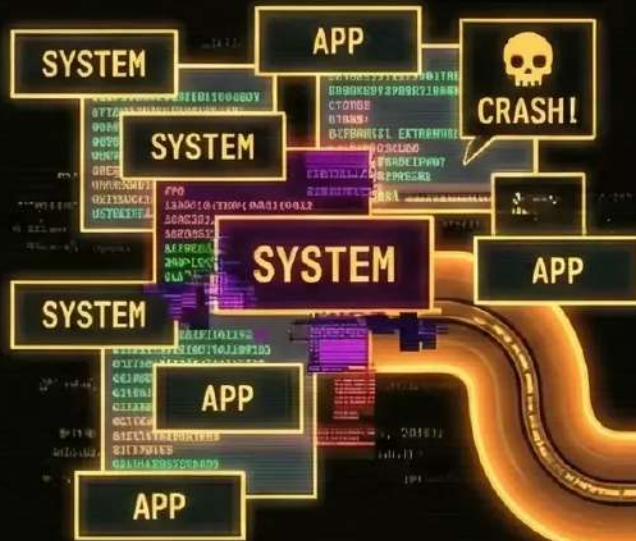
REAL MODE (16-BIT)



1MB RAM LIMIT
(SEGMENT:OFFSET)

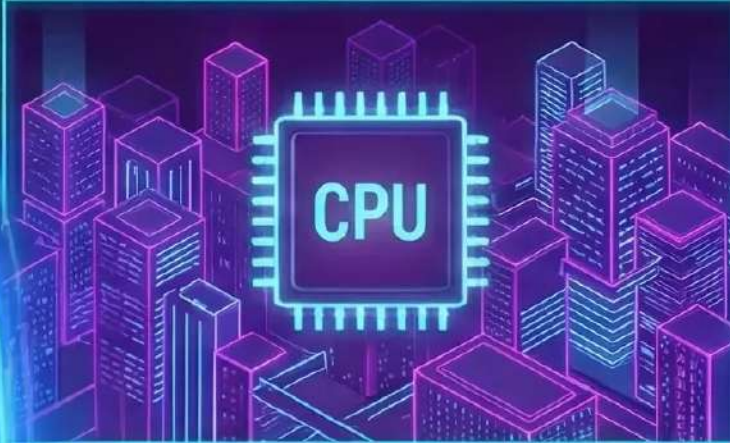
NO MEMORY PROTECTION
(WILD WEST)

SINGLE TASKING



MODE SWITCH
GATEWAY

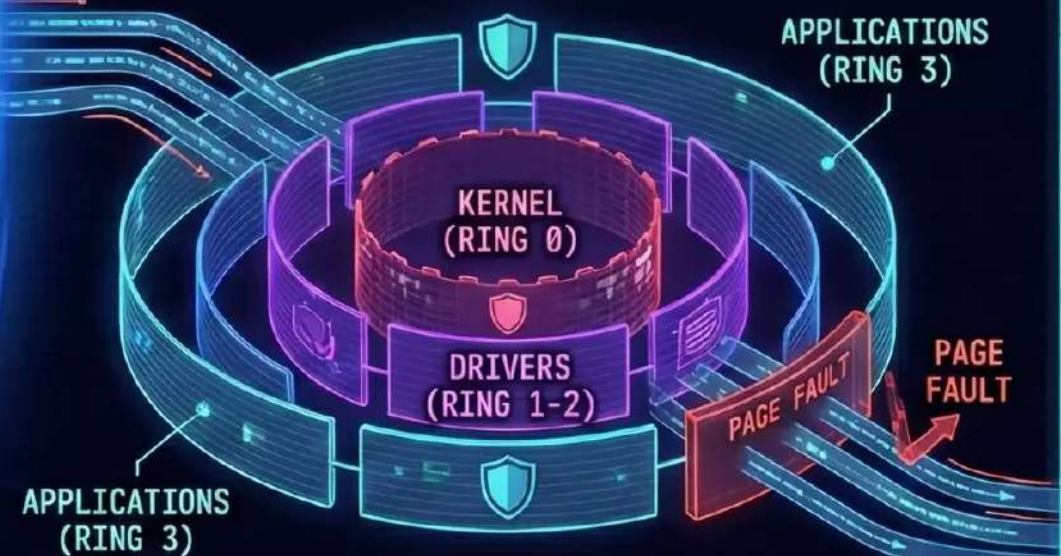
PROTECTED MODE (16/32-BIT+)



VIRTUAL MEMORY
(PAGING)

HARDWARE RINGS
(0-3 SECURITY)

MULTITASKING
ENABLED



>_ ACCESSING NEXT LEVEL ARCHITECTURE...

16-Bit x86 Register Set

Reg	Name	Primary Function in DEBUG context
AX	Accumulator	Primary arithmetic and logic; Input/Output operations; Return values.
BX	Base	Base pointer for memory access; High-order word of file size.
CX	Count	Loop counters; Low-order word of file size.
DX	Data	I/O port addressing; High-order word for multiplication/division.
SP	Stack Pointer	Pointer to the top of the stack (grows downwards).
BP	Base Pointer	Stack frame base pointer for accessing local variables.
SI	Source Index	Source pointer for string operations.
DI	Dest Index	Destination pointer for string operations.
CS	Code Segment	Segment containing the currently executing instructions.
DS	Data Segment	Default segment for data variables.
SS	Stack Segment	Segment containing the stack.
ES	Extra Segment	Auxiliary segment; Critical for INT 13h disk buffer pointers.
IP	Instruction Ptr	Offset of the next instruction to be executed.
FL	Flags	Status indicators (Zero, Carry, Overflow, Sign, etc.).

DOS Debug

- Debug "hello world"
- Flipfont

```
1 mov ah, 9
2 mov dx, 108
3 int 21
4 ret
5 'Hello world', D, A, '$'
```

```
1 E 100 FA BA C4 03 B8 02 04 EF B8 04 06 EF BA CE 03 B8
2 E 110 04 02 EF B8 05 00 EF B8 06 00 EF B8 00 A0 8E C0
3 E 120 31 FF B9 00 01 51 B9 08 00 89 FE 8D 5D 0F 26 8A
4 E 130 04 26 86 07 26 88 04 46 4B E2 F3 83 C7 20 59 E2
5 E 140 E4 BA C4 03 B8 02 03 EF B8 04 02 EF BA CE 03 B8
6 E 150 04 00 EF B8 05 10 EF B8 06 0E EF FB B8 00 4C CD
7 E 160 21
```



FreeDOS 

Box
SS


```
C:\> debug
```

```
-?
```

```
assemble      A [address]
compare       C range address
dump          D [range]
enter         E address [list]
fill          F range list
go            G [=address] [addresses]
hex           H value1 value2
input         I port
load          L [address] [drive] [firstsector] [number]
move          M range address
name          N [pathname] [arglist]
output        O port byte
proceed       P [=address] [number]
quit          Q
register       R [register]
search        S range list
trace         T [=address] [value]
unassemble    U [range]
write         W [address] [drive] [firstsector] [number]
allocate expanded memory  XA [#pages]
deallocate expanded memory XD [handle]
map expanded memory pages XM [Lpage] [Ppage] [handle]
display expanded memory status XS
```



Most Popular Languages

- **Sources:**

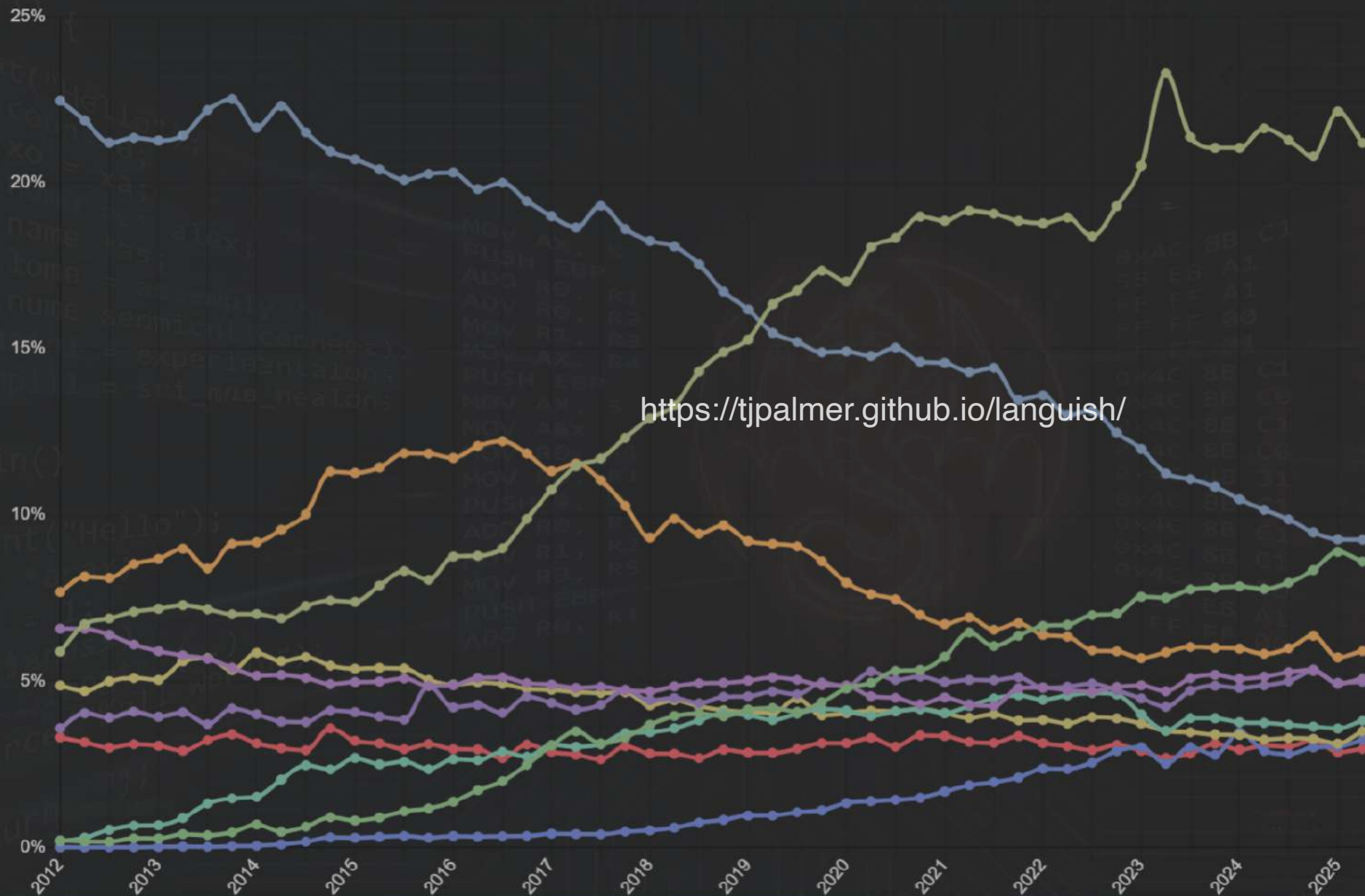
- **TIOBE:** tiobe.com/tiobe-index
- **Stack Overflow 2025 Survey:** survey.stackoverflow.co
- **IEEE:** spectrum.ieee.org/top-programming-languages-2025
- **GitHub Octoverse 2025:** github.blog/news-insights/octoverse/
- **PYPL:** pypl.github.io | github.com/pypl
- **Languish:** tjpalmer.github.io/languish | github.com/tjpalmer/languish
- **RedMonk:** redmonk.com/data/

Languish

Programming Language Trends
... for more, subscribe to Context Free



▼ Mean Score ▼



- 1 Python
- 2 JavaScript
- 3 TypeScript
- 4 Java
- 5 C#
- 6 C++
- 7 Go
- 8 HTML
- 9 Rust
- 10 C
- 11 PHP
- 12 CSS
- 13 Shell
- 14 Jupyter Notebook
- 15 Kotlin
- 16 R

Empty Reset Trim

Q

Data from GitHub and Stack Overflow

PYPL PopularitY of Programming Languages

Rank	Change	Language	Share	1-year trend
1		Python	24.61 %	-5.0 %
2	▲▲	C/C++	14.13 %	+6.9 %
3	▲▲▲▲▲	Objective-C	13.35 %	+10.5 %
4	▼▼	Java	10.45 %	-4.8 %
5	▲	https://delphi.org/rankings/PYPL.html	0.10 %	+1.6 %
6	▼▼▼	JavaScript	4.68 %	-3.3 %
7	▲▲▲▲	Swift	3.68 %	+1.1 %
8	▼	PHP	2.95 %	-0.9 %
9	▼▼▼▼▼	C#	2.79 %	-3.4 %
10	▲▲▲▲▲	Ada	2.71 %	+1.5 %

January 2026

© Pierre Carbonnelle

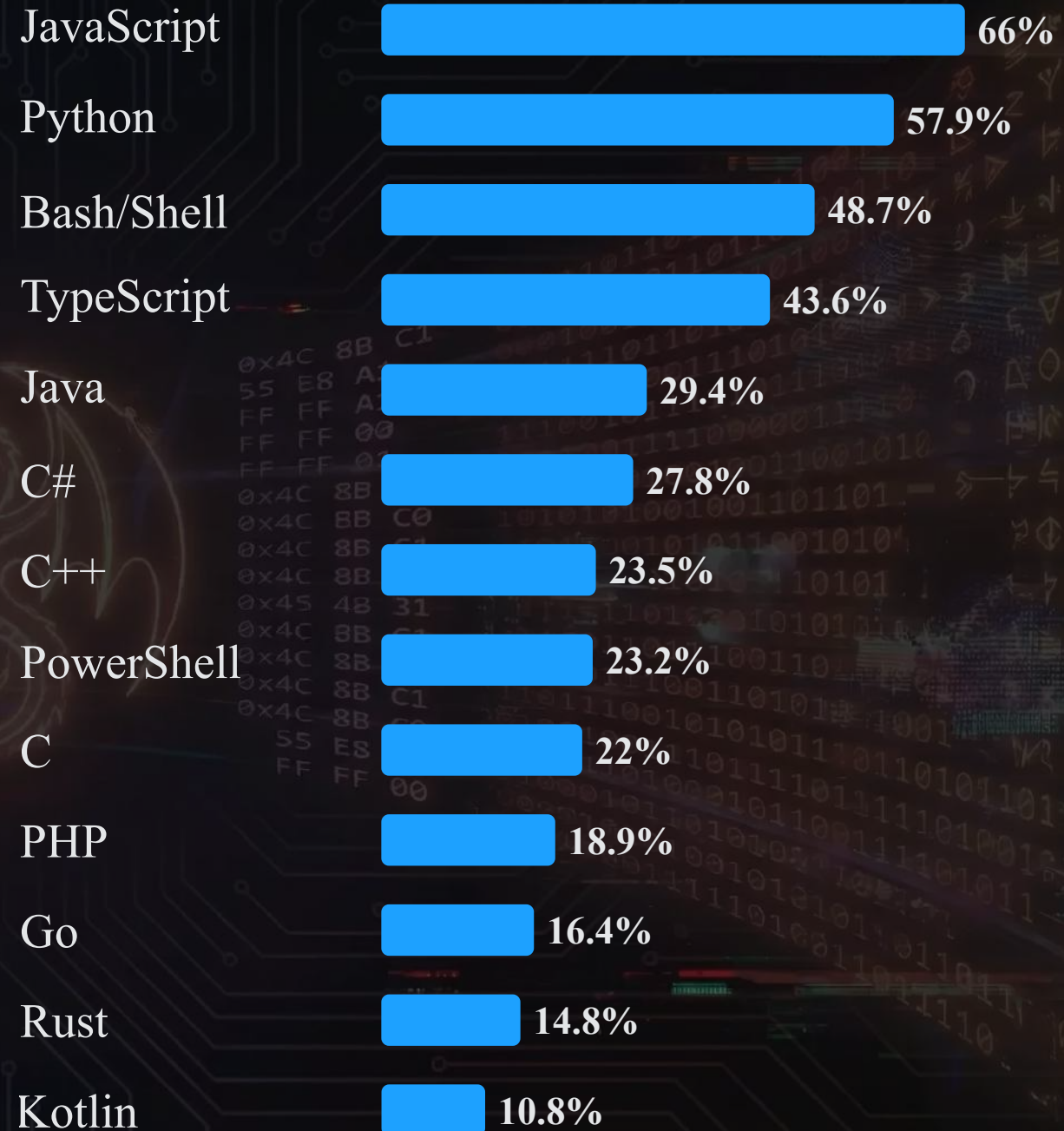
pypl.github.io

pypl.github.io | github.com/pypl

Determined by analyzing frequency of language tutorials are searched on Google

Stackoverflow

- 2025 Survey Results
- For "All Respondents"
- Responses: 31,771 (64.8%)
- (removed HTML & SQL)





TIOBE

- Ranking in January 2026

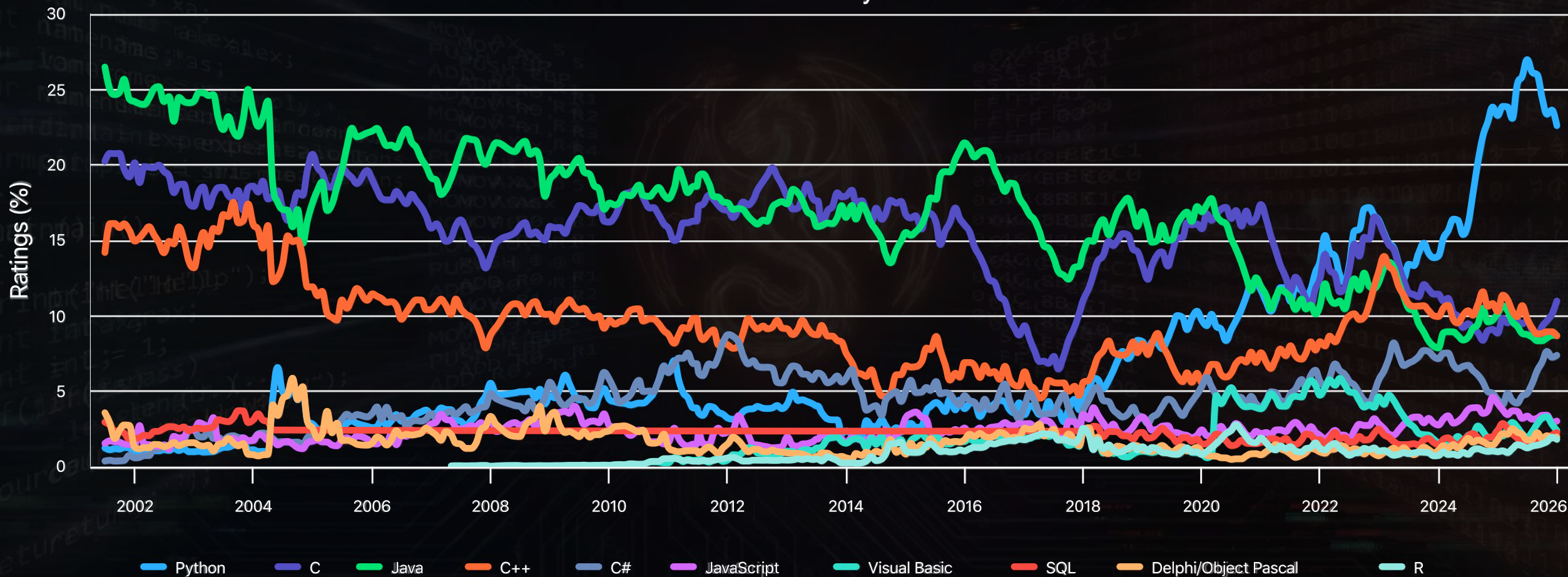
- Based on search data

Jan 2026	Jan 2025	Change	Programming Language	Ratings	Change
1	1		Python	22.61%	-0.68%
2	4	▲	C	10.99%	0.0213
3	3		Java	8.71%	-1.44%
4	2	▼	C++	8.67%	-1.62%
5	5		C#	7.39%	0.0294
6	6		JavaScript	3.03%	-1.17%
7	9		Visual Basic	2.41%	0.0004
8	8		SQL	2.27%	-0.14%
9	11	▲	Delphi/Object Pascal	1.98%	0.0019
10	18	▲▲	R	1.82%	0.0081
11	32	▲▲	Perl	1.63%	0.0114
12	10	▼	Fortran	1.61%	-0.42%
13	14	▲	Rust	1.51%	0.0034
14	15	▲	MATLAB	1.40%	0.0034
15	13	▼	PHP	1.38%	0.00%
16	7	▼▼	Go	1.24%	-1.37%
17	12	▼▼	Scratch	1.24%	-0.31%
18	26	▲▲	Ada	1.19%	0.0054
19	17	▼	Assembly language	1.07%	0.0005
20	25	▲▲	Kotlin	0.97%	0.0023



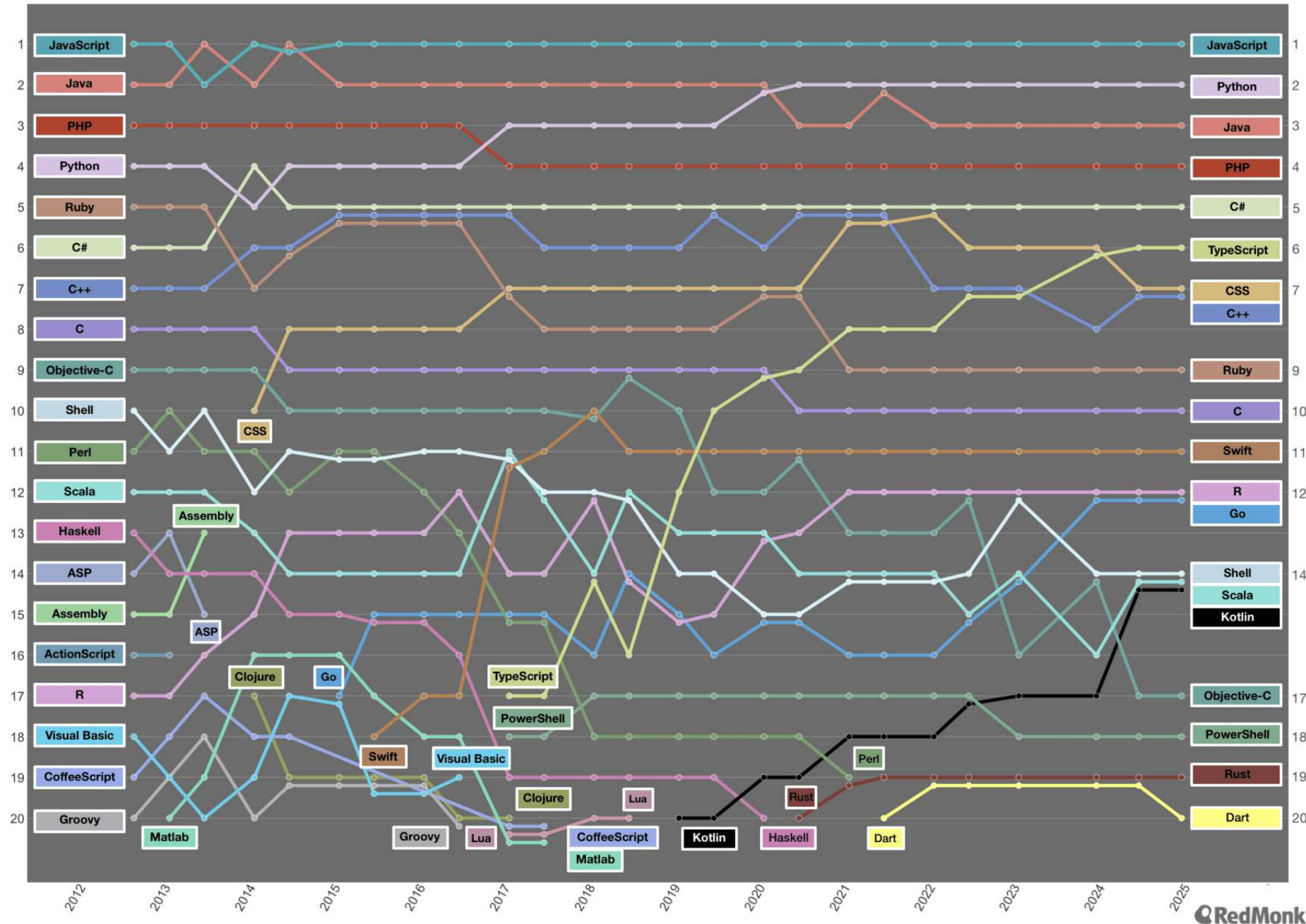
TIOBE

January 2026



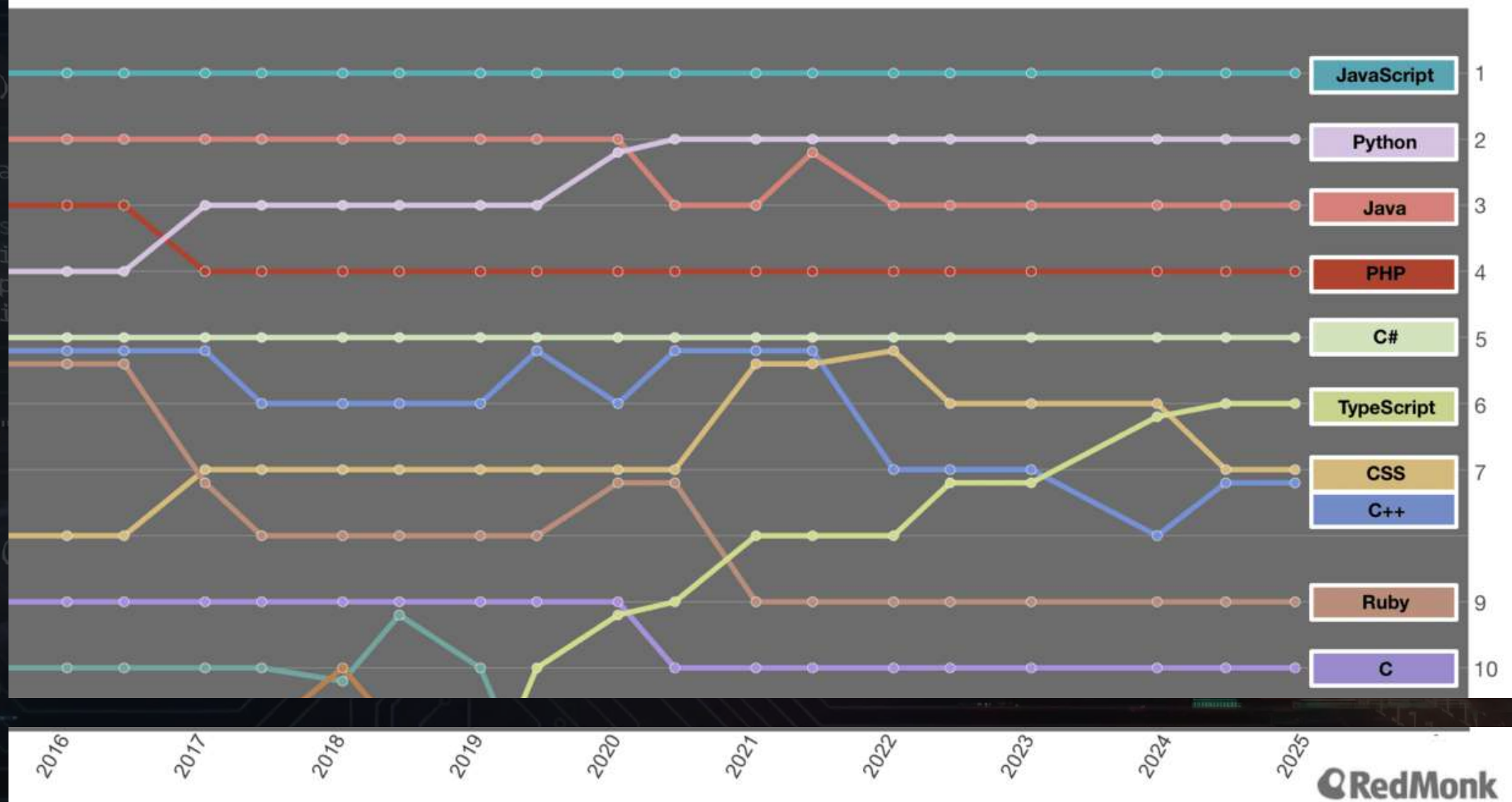
RedMonk Language Rankings

September 2012 - December 2024



RedMonk Language Rankings

September 2012 - December 2024



Overall

Top 10ish

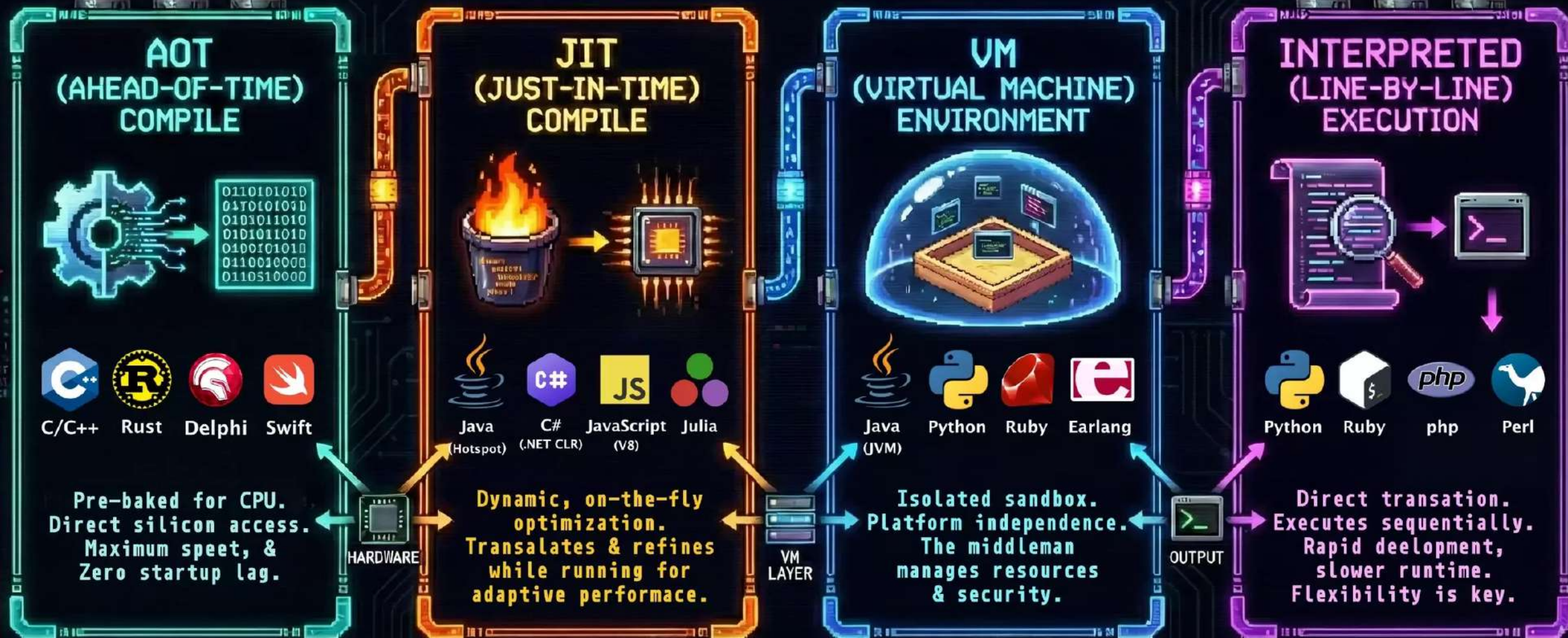
- JavaScript/TypeScript
- Python
- Java
- C#
- PHP
- C/C++
- Ruby
- Bash/Shell/PowerShell

Other Common

- Ada
- Rust
- Go
- Kotlin
- Swift
- R
- Objective-C
- Visual Basic
- Delphi/Pascal



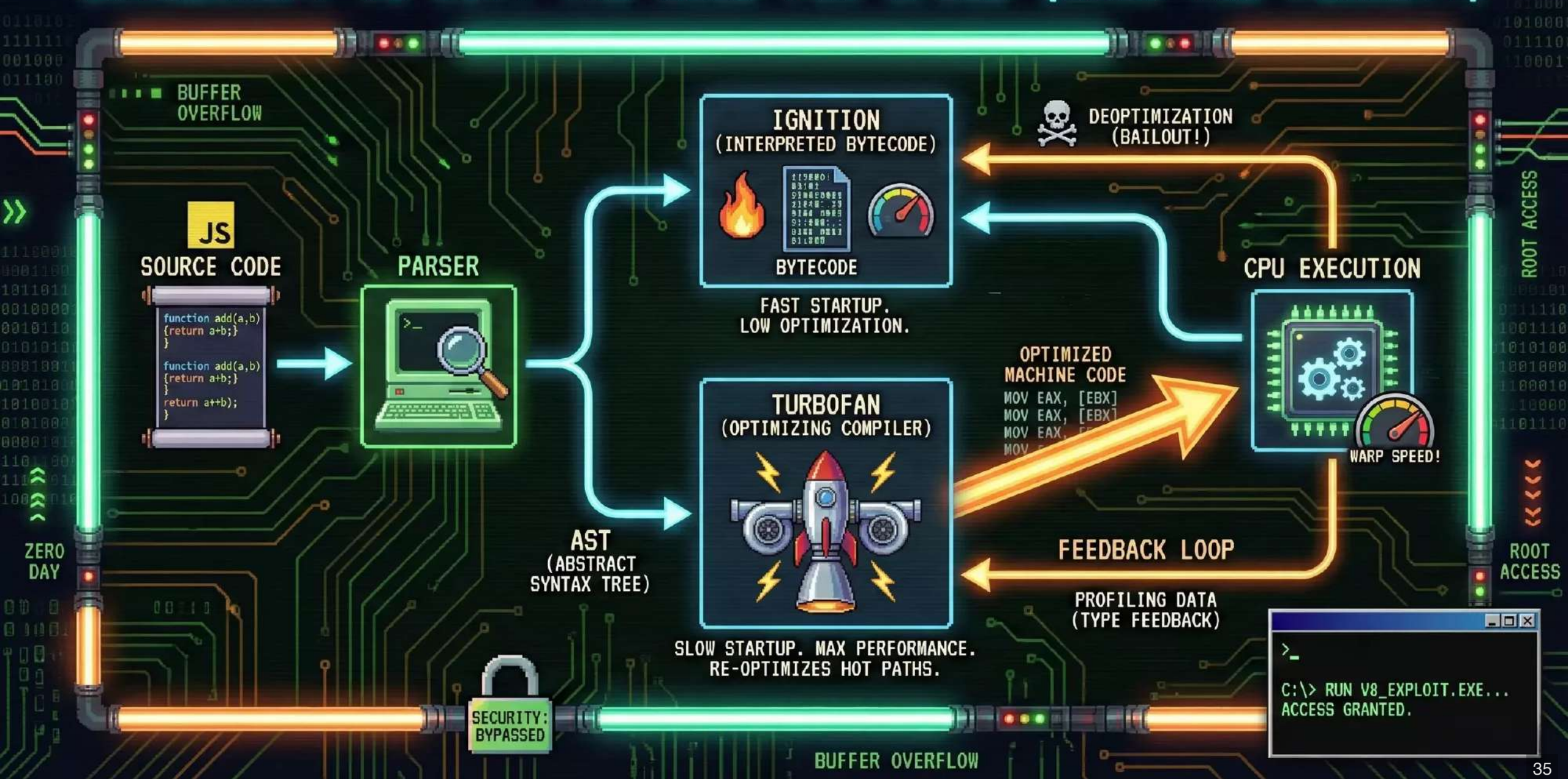
EXECUTION MODELS:



A dark, textured background with a glowing green 'JavaScript' text in the center. The background features a circuit board pattern, binary code, and faint code snippets in various colors (green, blue, red). The text 'JavaScript' is large, bold, and has a slight glow. The background elements include a circuit board pattern, binary code, and faint code snippets in various colors (green, blue, red). The overall aesthetic is tech-oriented and digital.

JavaScript Debugging

```
1 console.log("The basic log");
2
3 // Group related logs to keep the console tidy
4 console.group("User Transaction");
5 console.log("User: Alice");
6 console.log("Item: Sword of Truth");
7 console.groupEnd();
8
9 // Display arrays of objects as a clean table
10 const users = [
11   { name: "Alice", role: "Mage", hp: 100 },
12   { name: "Bob", role: "Warrior", hp: 150 }
13 ];
14 console.table(users);
15
16 // Only logs if the condition is false
17 console.assert(users[0].hp > 0, "User is dead!", users[0]);
18
19 // Print a stack trace
20 console.trace();
```


[illegible]

JavaScript Disassembly

```
1 function square(a) {  
2     let result = a * a;  
3     return result;  
4 }  
5  
6 // Collect type information on next call of function  
7 %PrepareFunctionForOptimization(square)  
8  
9 // Call function once to fill type information  
10 square(23);  
11  
12 // Call function again to go from uninitialized  
13 //      -> pre-monomorphic -> monomorphic  
14 square(13);  
15 %OptimizeFunctionOnNextCall(square);  
16 square(71);
```


JavaScript Disassembly

godbolt.org/z/MhKG9vvej

```
1  --- Raw source ---
2  (a) {
3      let result = a * a;
4      return result;
5  }
```

```
7  --- Optimized code ---
8  optimization_id = 0
9  source_position = 15
10 kind = TURBOFAN
11 name = square
12 stack_slots = 6
13 compiler = turbofan
14 address = 0x3b0800002259
```

```
16 Instructions (size = 284)
17 0x5d99e0004040    0 8b59f4
18 0x5d99e0004043    3 4d8b95d0010000
19 0x5d99e000404a    a 490bda
20 0x5d99e000404d    d f6431a20
21 0x5d99e0004051   11 0f85293ddea0
22 0x5d99e0004057   17 55
23 0x5d99e0004058   18 4889e5
24 0x5d99e000405b   1b 56
25 0x5d99e000405c   1c 57
26 0x5d99e000405d   1d 50
27 0x5d99e000405e   1e 4883ec08
28 0x5d99e0004062   22 488975e0
29 0x5d99e0004066   26 493b65a0
```

```
movl rbx,[rcx-0xc]
Rex.W movq r10,[r13+0x1d0]
Rex.W orq rbx,r10
testb [rbx+0x1a],0x20
jnz 0x5d9980de7d80 (CompileLazyDeoptimizedCode) ;; near builtin entry
push rbp
Rex.W movq rbp,rsi
push rsi
push rdi
push rax
Rex.W subq rsp,0x8
Rex.W movq [rbp-0x20],rsi
Rex.W cmpq rsp,[r13-0x60] (external value (StackGuard::address_of_jslimit()))
```

Inlined functions (count = 0)

Deoptimization Input Data (deopt points = 4)

index	bytecode-offset	pc
0	2	NA
1	2	NA
2	2	NA
3	-1	b7

Safepoints (entries = 2, byte size = 20)

0x5d99e00040f7	b7	slots (sp->fp): 10000000	deopt	3 trampoline:	118
0x5d99e0004122	e2	slots (sp->fp): 00000000			

RelocInfo (size = 9)

0x5d99e0004053	near builtin entry
0x5d99e00040ea	full embedded object (0x10a400183c85 <NativeContext[285]>)
0x5d99e00040f3	near builtin entry
0x5d99e000411e	near builtin entry



VS.



GCC vs LLVM

Compiler Infrastructure Platforms



GCC vs LLVM



	GCC	LLVM
Initial release	March 22, 1987; 38 years ago	2003; 23 years ago
Original authors	Richard Stallman	Chris Lattner, Vikram Adve
Stable release	15.2 / 8 Aug 2025	21.1.8 / 16 Dec 2025
Written in	C, C++	C++
Languages	49	27
OS	Cross-platform	Cross-platform
Size	~15 million LOC	~35.56 million LOC
License	GPLv3+ with GCC Runtime Library Exception	Apache License 2.0 with LLVM Exceptions

LLVM VS GCC

Feature	GCC	LLVM
Primary License	GPLv3 (Copyleft). Ensures derivative works remain open-source.	Apache 2.0 with LLVM Exceptions (Permissive). Allows for proprietary commercial extensions.
Architecture	Monolithic. Older design with tightly coupled components. Difficult to use its internal parts as separate libraries.	Modular. Designed as a set of reusable libraries with clear interfaces. Easy to integrate into other tools.
Intermediate Representation (IR)	Uses GIMPLE and RTL internally. These are not designed for external use or tooling.	Uses LLVM IR, a universal, language-independent, assembly-like language designed for analysis and optimization by external tools.
Primary Frontends	gcc (C), g++ (C++), gfortran (Fortran), Ada, Go, D, Objective-C.	Clang is the primary frontend for C, C++, and Objective-C. Used as a backend by Rust, Swift, Julia, Ruby, etc.
Platform Support	Extremely broad, including many legacy and niche architectures. The standard for Linux and embedded systems.	Broad support for modern architectures (x86, ARM, RISC-V, Wasm). The default on macOS, iOS, and widely used in Android NDK.
Compilation Speed	generally slower, especially at higher optimization levels.	Generally faster, particularly with the Clang frontend. Known for fast, incremental builds.
Generated Code Performance	Mature and highly optimized. Historically had a slight edge in some CPU-intensive benchmarks.	Highly competitive. Often matches or beats GCC, especially with modern features like advanced vectorization and Link-Time Optimization (LTO).
Diagnostics & Tooling	Traditional error messages. Tooling includes GDB and GNU binutils.	Superior, human-readable error messages. Rich ecosystem of modular tools like LLDB, Clang Static Analyzer, clang-tidy, and clang-format.
Key Strengths	Maturity, stability, incredibly wide hardware support, and strict adherence to open-source principles.	Modularity, build speed, excellent diagnostics, modern design, and permissiveness for commercial use.

GCC, the GNU Compiler Collection

The GNU Compiler Collection includes front ends for [C](#), [C++](#), Objective-C, Objective-C++, [Fortran](#), Ada, Go, D, Modula-2, COBOL, Rust, and Algol 68 as well as libraries for these languages (libstdc++,...). GCC was originally written as the compiler for the [GNU operating system](#). The GNU system was developed to be 100% free software, free in the sense that it [respects the user's freedom](#).

We strive to provide regular, high quality [releases](#), which we want to work well on a variety of native and cross targets (including GNU/Linux), and encourage everyone to [contribute](#) changes or help [testing](#) GCC. Our sources are readily and freely available via [Git](#) and weekly [snapshots](#).

Major decisions about GCC are made by the [steering committee](#), guided by the [mission statement](#).



FKA "GNU C Compiler"
<https://gcc.gnu.org/>

News

GCC developer room at FOSDEM 2026: Schedule Available
[2025-12-15]

FOSDEM 2026: Brussels, Belgium, Jan 31 - Feb 1, 2026

Algol 68 front end added [2025-11-30]

An experimental Algol 68 programming language front end has been added to GCC.

GCC developer room at FOSDEM 2026: Call for Participation [2025-11-03]

FOSDEM 2026: Brussels, Belgium, Jan 31 - Feb 1, 2026

GCC 15.2 released [2025-08-08]

GNU Tools Cauldron 2025 [2025-08-01]

Porto, Portugal, September 26-28 2025

GCC 12.5 released [2025-07-11]

GCC 13.4 released [2025-06-05]

GCC 14.3 released [2025-05-23]

GCC 15.1 released [2025-04-25]

COBOL front end added [2025-04-17]

The COBOL programming language front end has been added to GCC. This front end was contributed by COBOLworx.

[Older news](#) | More news? Let gerald@pfeifer.com know!

Supported Releases

GCC 15.2 ([changes](#))

Status: **2025-08-08** (regression fixes & docs only).

[Serious regressions](#). [All regressions](#).

GCC 14.3 ([changes](#))

Status: **2025-05-23** (regression fixes & docs only).

[Serious regressions](#). [All regressions](#).

GCC 13.4 ([changes](#))

Status: **2025-06-05** (regression fixes & docs only).

[Serious regressions](#). [All regressions](#).

Development: GCC 16.0 ([release criteria](#), [changes](#))

Status: **2026-01-12** (regression fixes & docs only).

[Serious regressions](#). [All regressions](#).

Search our site

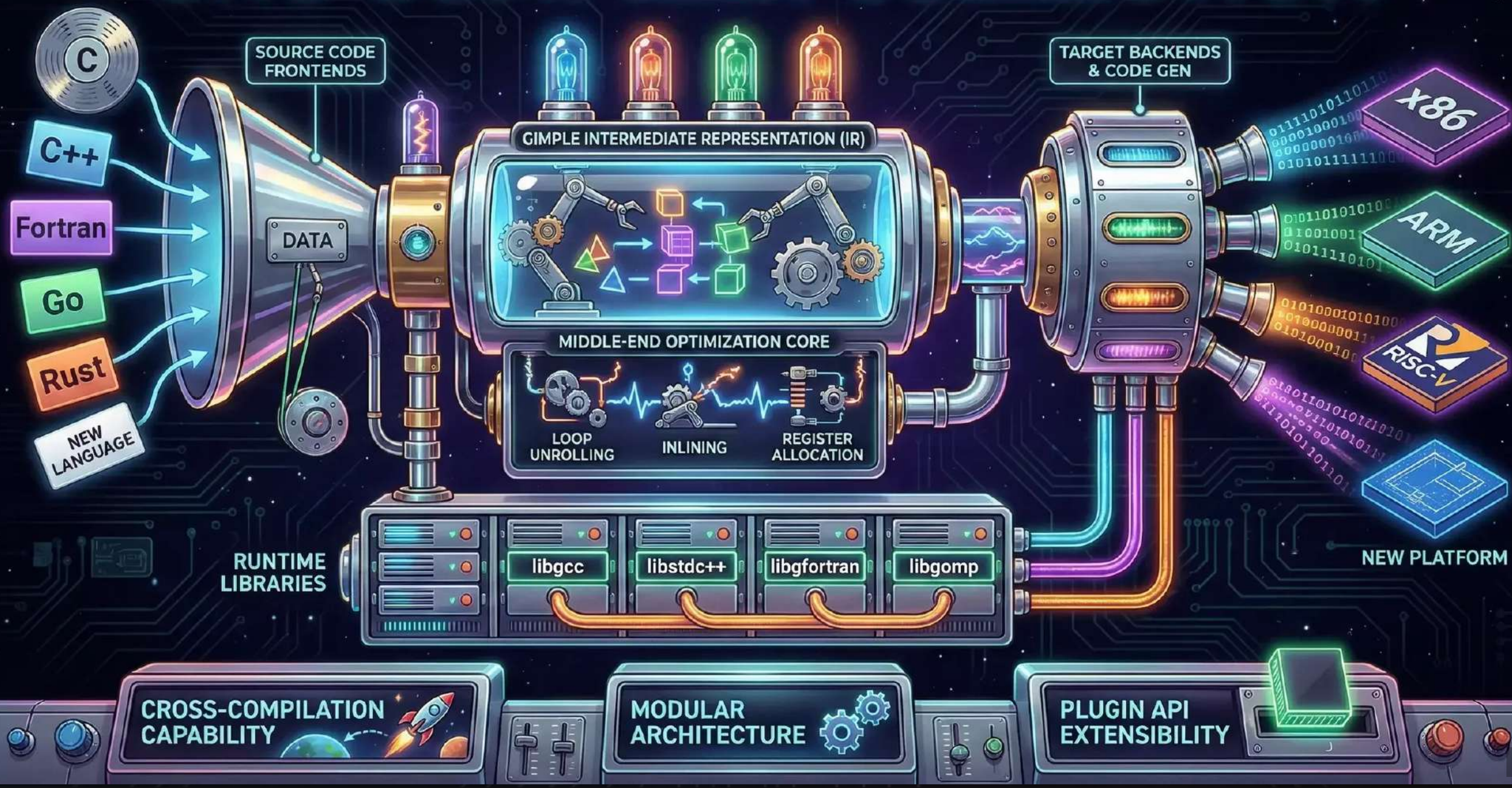
Match: Sort by:

There is also a [detailed search form](#).

Get our announcements

Initial release	March 22, 1987; 38 years ago
Original author	Richard Stallman
Stable release	15.2 / 8 Aug 2025
Written in	C, C++
Languages	49
OS	Cross-platform
Size	~15 million LOC
License	GPLv3+ with GCC Runtime Library Exception

GNU COMPILER COLLECTION



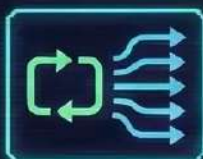


GCC COMPILER OPTIMIZATIONS



MAXIMIZING CODE PERFORMANCE THROUGH ADVANCED TRANSFORMATIONS

LOOP UNROLLING



REDUCES LOOP OVERHEAD BY EXECUTING MULTIPLE ITERATIONS IN A SINGLE STEP.

BEFORE

```
for (i=0; i<100; i++) {  
    ...  
}
```

AFTER

```
for (i=0; i<100; i++) {  
    ...  
    i=0, 4, 8...  
    i=1, 5, 9...  
    i=2, 6, 10...  
    i=3, 7, 11...  
}
```

INCREASED INSTRUCTION LEVEL PARALLELISM, FASTER EXECUTION.

INLINING



ELIMINATES FUNCTION CALL OVERHEAD BY INSERTING THE FUNCTION'S CODE DIRECTLY.

BEFORE

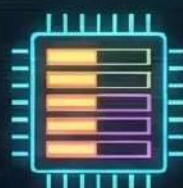
```
main() {  
    foo()  
}  
foo() {  
    ...  
}
```

AFTER

```
main() {  
    foo() {  
        ...  
    }  
}
```

AVIODS CONTEXT SWITCHING, IMPROVES CACHE LOCALITY.

REGISTER ALLOCATION



MINIMIZES MEMORY ACCESS BY KEEPING FREQUENTLY USED VARIABLES IN CPU REGISTERS.

BEFORE

MEMORY

a
b
c
d

AFTER

CPU

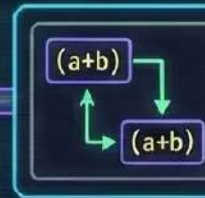
a	R1
b	R2
c	R3
d	R4

FASTER DATA ACCESS, REDUCED LATENCY.

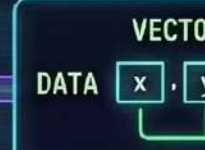
OTHER OPTIMIZATIONS



DEAD CODE ELIMINATION
REMOVES UNREACHABLE OR UNUSED CODE SECTIONS.



COMMON SUBEXPRESSION ELIMINATION
AVIODS REDUNDANT COMPUTATIONS.



EXPLOITS SIMD (SINGLE INSTRUCTION, MULTIPLE DATA) CAPABILITIES.

GCC: UNLEASHING PERFORMANCE THROUGH INTELLIGENT COMPIATION. OPTIMIZE RESPONSIBLY.

The LLVM Compiler Infrastructure

Site Map:

[Overview](#)
[Features](#)
[Documentation](#)
[Command Guide](#)
[FAQ](#)
[Publications](#)
[LLVM Projects](#)
[Open Projects](#)
[LLVM Users](#)
[Bug tracker](#)
[LLVM Logo](#)
[Blog](#)
[Meetings](#)
[LLVM Foundation](#)

Download!

Download now:
[LLVM 21.1.8](#)
[All Releases](#)
[APT Packages](#)
[RPM Snapshots](#)
[Pre-releases](#)

View the open-source
[license](#)

Search this Site

LLVM Overview

The LLVM Project is a collection of modular and reusable compiler and toolchain technologies. Despite its name, LLVM has little to do with traditional virtual machines. The name "LLVM" itself is not an acronym; it is the full name of the project.

LLVM began as a [research project](#) at the [University of Illinois](#), with the goal of providing a modern, SSA-based compilation strategy capable of supporting both static and dynamic compilation of arbitrary programming languages. Since then, LLVM has grown to be an umbrella project consisting of a number of subprojects, many of which are being used in production by a wide variety of [commercial and open source](#) projects as well as being widely used in [academic research](#). Code in the LLVM project is licensed under the ["Apache 2.0 License with LLVM exceptions"](#)

The primary sub-projects of LLVM are:

1. The **LLVM Core** libraries provide a modern source- and target-independent [optimizer](#), along with [code generation support](#) for many popular CPUs (as well as some less common ones!) These libraries are built around a [well specified](#) code representation known as the LLVM intermediate representation ("LLVM IR"). The LLVM Core libraries are [well documented](#), and it is particularly easy to invent your own language (or port an existing compiler) to use [LLVM as an optimizer and code generator](#).
2. **Clang** is an "LLVM native" C/C++/Objective-C compiler, which aims to deliver amazingly fast compiles, extremely useful [error and warning messages](#) and to provide a platform for building great source level tools. The [Clang Static Analyzer](#) and [clang-tidy](#) are tools that automatically find bugs in your code, and are great examples of the sort of tools that can be built using the Clang frontend as a library to parse C/C++ code.
3. The **LLDB** project builds on libraries provided by LLVM and Clang to provide a great native debugger. It uses the Clang ASTs and expression parser, LLVM JIT, LLVM disassembler, etc so that it provides an experience that "just works". It is also blazing fast and much more memory efficient than GDB at loading symbols.
4. The [libc++](#) and [libc++ ABI](#) projects provide a standard conformant and high-performance implementation of the C++ Standard Library, including full support for C++11 and C++14.
5. The [libc](#) project provides a high-performance, standards-conformant implementation of the C Standard Library.

Latest LLVM Release!

16 December 2025: LLVM 21.1.8 is now [available for download!](#) LLVM is publicly available under an open source [License](#). Also, you might want to check out [the new features](#) in Git that will appear in the next LLVM release. If you want them early, [download LLVM](#) through anonymous Git.

Upcoming Events

[2026 EuroLLVM Developers' Meeting](#)

ACM Software System Award!

LLVM has been awarded the **2012 ACM Software System Award!** This award is given by ACM to *one* software system worldwide every year. LLVM is [in highly distinguished company!](#) Click on any of the individual recipients' names on that page for the detailed citation describing the award.

Upcoming Releases

LLVM Release Schedule:

- 22.1.x
 - Tue Jan 13th, 2026: release/22.x branch
 - Fri Jan 16th, 2026: 22.1.0-rc1
 - Tue Jan 27th, 2026: 22.1.0-rc2
 - Tue Feb 10th, 2026: 22.1.0-rc3
 - Tue Feb 24th, 2026: 22.1.0

LLVM LANGUAGES (Frontends)

C/C++
(Clang)



Rust



Swift



Zig



julia

Kotlin/Native



TARGET PLATFORMS (Backends)



x86
(Desktop/Server)



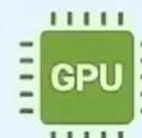
ARM
(Mobile/IoT)



WebAssembly
(Web)



RISC-V
(Embedded)



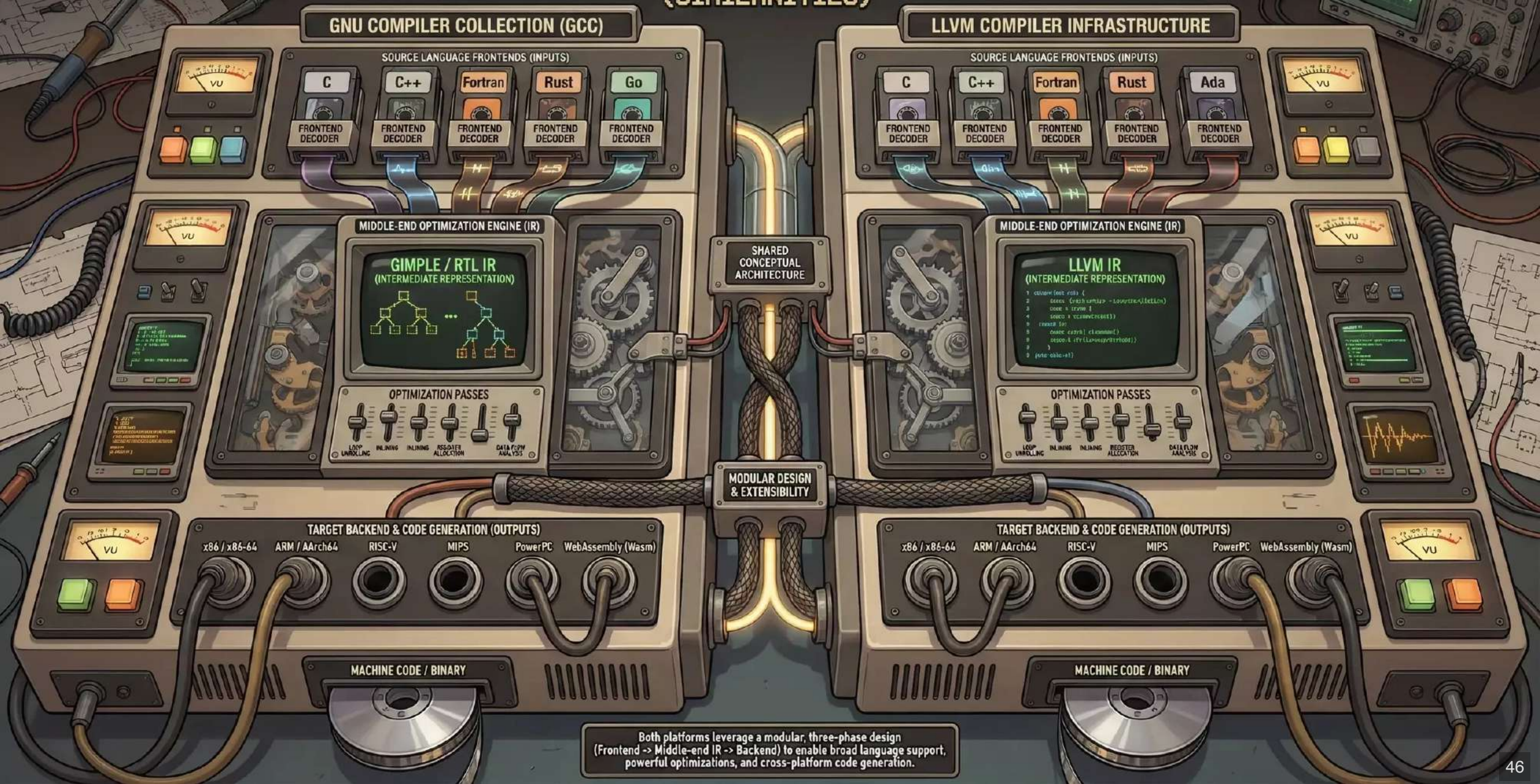
GPU
(CUDA/ROCm)



BPF
(Linux Kernel)

GCC & LLVM: COMPILER INFRASTRUCTURE PLATFORMS

(SIMILARITIES)



GCC

(GNU COMPILER COLLECTION)

LICENSE & GOVERNANCE



GPL

GENERAL PUBLIC LICENSE



FREE SOFTWARE
FOUNDATION

COPYLEFT, STRICTER INTEGRATION, COMMUNITY-DRIVEN

ARCHITECTURE & IR (GIMPLE/RTL)

GIMPLE / RTL IR



MULTIPLE, COMPLEX IRs,
TIGHTLY COUPLED TO BACKEND

MONOLITHIC DESIGN

ECOSYSTEM & TOOLING

INTEGRATED TOOLCHAIN

GCC

GDB

BINUTILS

GLIBC

MATURE, INTEGRATED SUITE, EXTENSIVE PLATFORM SUPPORT

GCC vs. LLVM: KEY DIFFERENCES

LLVM

(COMPILER INFRASTRUCTURE)

LICENSE & GOVERNANCE



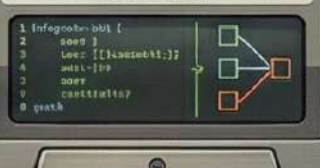
APACHE 2.0 / PERMISSIVE

PERMISSIVE, FLEXIBLE REUSE, INDUSTRY-BACKED

LLVM
FOUNDATION

ARCHITECTURE & IR (LLVM IR)

LLVM IR



SINGLE, UNIFIED, SSA-BASED IR,
SEPARABLE COMPONENTS

MODULAR & DECOUPLED

MODULAR & DECOUPLED

ECOSYSTEM & TOOLING

CLANG

LLDB

LLD

LIBC++

MLIR

MODULAR TOOLS, REUSABLE LIBRARIES, RAPID INNOVATION

While both platforms are powerful, GCC's monolithic, copyleft nature contrasts with LLVM's modular, permissive architecture, leading to different use cases, integration strategies, and community dynamics.

LLVM



Compiler Infrastructure

llvm.org

LLVM: THE UNIVERSAL CODE ENGINE

C/C++ (Clang)
SYSTEMS & PERFORMANCE



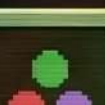
RUST
SAFETY & CONCURRENCY



SWIFT
APP DEVELOPMENT



JULIA
SCIENTIFIC COMPUTING



ZIG
ROBUST & OPTIMIZED

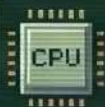


**OTHER
FRONTENDS**



**LLVM CORE
(IR)**

**LANGUAGE-AGNOSTIC
INTERMEDIATE REPRESENTATION
& OPTIMIZATION PIPELINE**



x86/x64
DESKTOP & SERVER



ARM
MOBILE & EMBEDDED



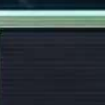
WEBASSEMBLY
WEB BROWSERS



GPU (NVPTX/AMDGPU)
PARALLEL PROCESSING

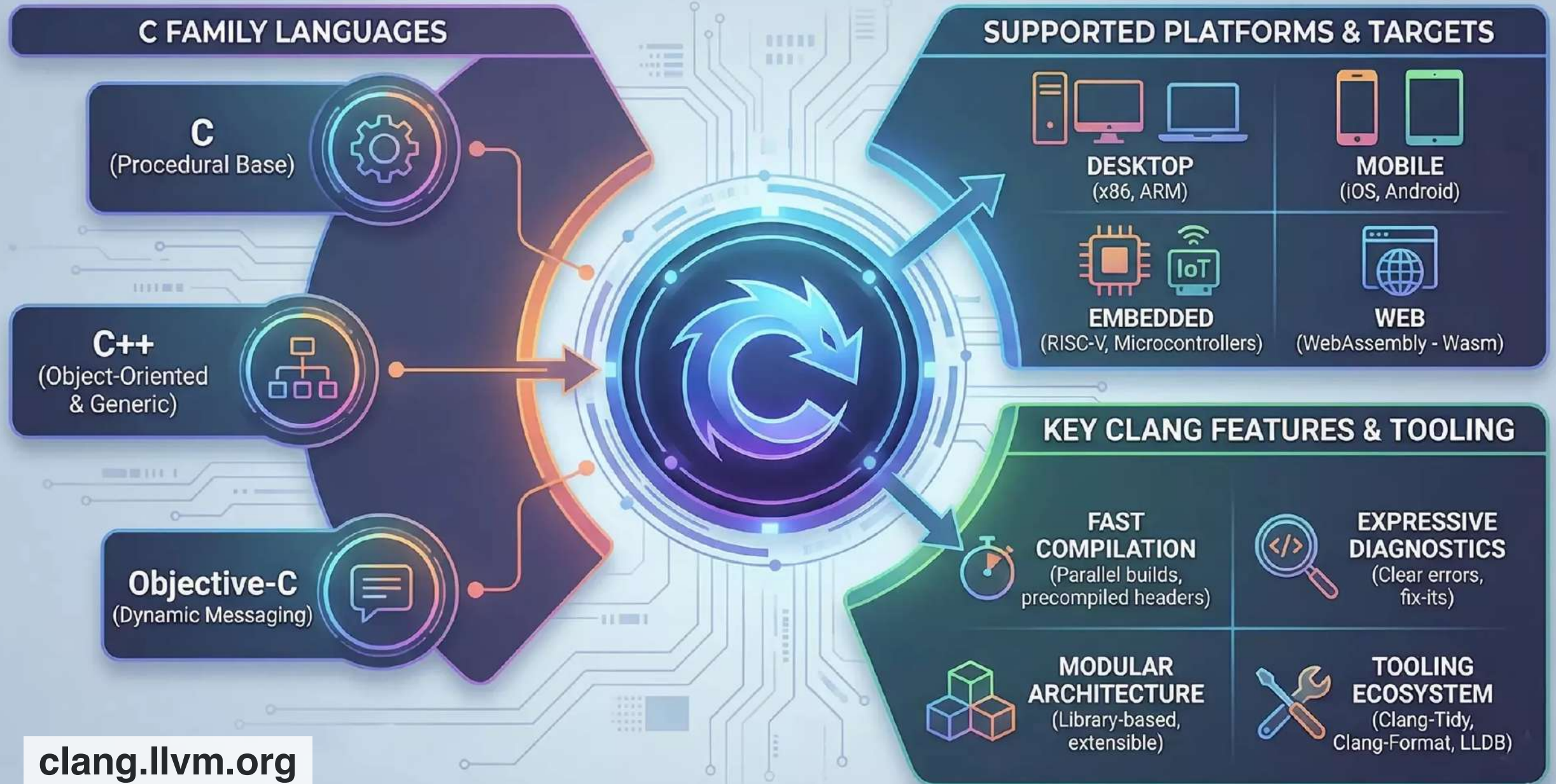


RISC-V
OPEN ISA



**OTHER
BACKENDS**

CLANG & THE C FAMILY: COMPILER, PLATFORMS, FEATURES

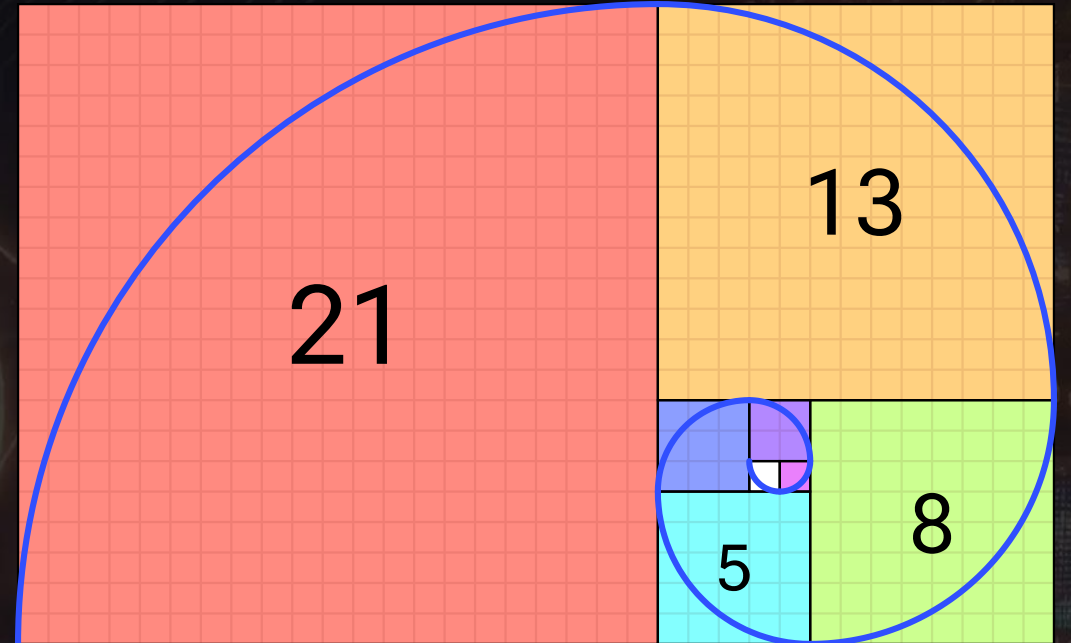


clang.llvm.org

Code Examples

- Iterative Fibonacci sequence
- Sum of the two preceding elements
- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

$$\begin{aligned} F_0 &= 0, & F_1 &= 1, \\ [F_n &= F_{n-1} + F_{n-2} \\ & n > 1] \end{aligned}$$



https://en.wikipedia.org/wiki/Fibonacci_sequence

Fibonacci spiral by Randaum

C source #1

A ▾ Save/Load + Add new... ▾ Vim

C

x86-64 gcc 15.2 (Editor #1)

x86-64 gcc 15.2

Compiler options...

```

1 #include <stdio.h>
2
3 unsigned long long fib_iterative(int n) {
4     if (n == 1) return 1;
5     unsigned long long a = 0; // F(0)
6     unsigned long long b = 1; // F(1)
7     unsigned long long next;
8     for (int i = 2; i <= n; i++) {
9         a = b;
10        b = next;
11        next = a + b;
12    }
13    return b;
14 }
15
16 int main() {
17     int n = 10;
18     unsigned long long result = fib_iterative(n);
19     printf("%llu is the nth Fibonacci number at position %d\n", result, n);
20     return 0;
21 }

```

(endColumn:2,endLineNumber:30,positionColumn:2,positionLineNumber:30,selectionStartColumn:2,selectionStartLineNumber:30,startColumn:2,startLineNumber:30),source:'%23include+%3Cstdio.h%3E%0A%0A//+Function+to+calculate+the+nth+Fibonacci+number%0Aunsigned+long+long+fib_iterative(int+n)%7B%0A++++if+(n+%3C%3D+0)+return+0%3B%0A++++if+(n+%3D%3D+1)+return+1%3B%0A%0A++++unsigned+long+long+a+%3D+0%3B+//+F(0)%0A++++unsigned+long+long+b+%3D+1%3B+//+F(1)%0A++++unsigned+long+long+next%3B%0A%0A++++//+Iterate+from+2+up+to+n%0A++++for+(int+i+%3D+2;+i+<=+n;+i++)+%7B%0A++++next+%3D+a+b%3B%0A++++a+%3D+b%3B%0A++++b+%3D+next%3B%0A++++%7D%0A%0A++++return+b%3B%0A%7D%0A%0Aint+main()+%7B%0A++++int+n+%3D+10%3B%0A++++unsigned+long+long+result+%3D+fib_iterative(n)%3B%0A%0A++++//+%25llu+is+the+format+specifier+for+unsigned+long+long%0A++++printf("%22Fibonacci+number+at+position+%25d+is:+%25llu%5Cn%22,+n,+result)%3B%0A%0A++++return+0%3B%0A%7D%0A%0A',l:'5',n:'0',o:'C+source+%231',t:'0'))',k:50,l:'4',n:'0',o:',s:0,t:'0'))',(g:(h:compiler,i:(compiler:cg152,filters:(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0')),flagsViewOpen:'1',fontScale:14,fontUsePx:'0',j:1,lang:'c,libs:!(,options:',overrides:!(,selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'+x86-64+gcc+15.2+(Editor+%231)',t:'0'))',k:50,l:'4',n:'0',o:',s:0,t:'0'))',l:'2',m:100,n:'0',o:',t:'0'))',version:4

C++ source #1

A ▾ Save/Load + Add new... ▾ Vim CppInsights Quick-bench

```
1 #include <iostream>
2 #include <cstdint>
3 // Function to calculate the nth Fibonacci number
4 uint64_t fib_iterative(int n) {
5     if (n <= 0) return 0;
6     if (n == 1) return 1;
7     uint64_t a = 0; // F(0)
8     uint64_t b = 1; // F(1)
9     // Loop starts at 2 because we already defined 0 and 1
10    for (int i = 2; i <= n; ++i) {
11        uint64_t temp = a + b;
12        a = b;
13        b = temp;
14    }
15    return b;
16 }
17 int main() {
18     int n = 10;
19     // std::cout handles types automatically (no %llu needed)
20     std::cout << "The " << n << "th Fibonacci number is: "
21               << fib_iterative(n) << std::endl;
22     return 0;
23 }
```

<https://godbolt.org/z/qqx3T4cPf>

x86-64 gcc 15.2 (Editor #1)

A ▾ Output... ▾ Filter... ▾ Libraries Overrides + Add new... ▾ Add tool... ▾

```
1 fib_iterative(int):
2     mov     rbp, rsp
3     sub     rsp, 36
4     cmp     DWORD PTR [rbp-36], 0
5     jle     .L2
6     mov     eax, 0
7     jmp     .L2
8     .L2:
9     cmp     DWORD PTR [rbp-36], 1
10    jle     .L4
11    mov     eax, 1
12    .L4:
13    .L3:
14    .L4:
15    mov     rdx, rax
16    mov     QWORD PTR [rbp-16], 1
17    mov     rax, QWORD PTR [rbp-8]
18    jmp     .L5
19    .L5:
20    mov     rdx, QWORD PTR [rbp-8]
21    mov     rax, QWORD PTR [rbp-16]
22    add     rax, rdx
23    mov     QWORD PTR [rbp-16], rax
24    mov     rax, QWORD PTR [rbp-16]
25    mov     QWORD PTR [rbp-8], rax
26    mov     QWORD PTR [rbp-16], rax
27    add     DWORD PTR [rbp-20], 1
28    .L5:
29    mov     eax, DWORD PTR [rbp-36]
30    cmp     eax, DWORD PTR [rbp-36]
31    jle     .L3
32    jmp     .L4
33    .L6:
34    mov     rax, QWORD PTR [rbp-16]
35    pop     rbp
36    ret
37    .LC0:
```


RUST & LLVM:

FORGING THE FUTURE WITH C-FAMILY SYNERGY

RUST COMPILATION FLOW



RUST SOURCE
(.rs)

Rustc
FRONTEND
(HIR/MIR)

LLVM IR
(Intermediate Representation)

LLVM BACKEND CORE

OPTIMIZATION PIPELINE

CODE
GENERATION

C-FAMILY INTEGRATION & INTEROP (FFI)



C SOURCE
(.c)



C++ SOURCE
(.cpp)

FFI

(FOREIGN FUNCTION
INTERFACE) BRIDGE

Clang
FRONTEND

SEAMLESS ABI
COMPATIBILITY &
SHARED MEMORY

MACHINE CODE TARGETS

x86/x64

(DESKTOP/SERVER)

ARM

(MOBILE/EMBEDDED)

WASM

(WEBASSEMBLY)

THE UNIVERSAL TOOLCHAIN: UNIFYING RUST AND C FOR A SAFER, FASTER TOMORROW.

Rust source #1 

A ▾  Save/Load + Add new... ▾ Vim

 Rust

```
1 pub fn safe_fib(n: u32) -> Option<u64> {
2     if n == 0 { return Some(0); }
3     if n == 1 { return Some(1); }
4
5     let mut a: u64 = 0;
6     let mut b: u64 = 1;
7
8     for _ in 2..=n {
9         // If addition overflows, return None
10        let next = a.checked_add(b)?;
11        a = b;
12        b = next;
13    }
14    Some(b)
15 }
```

[https://godbolt.org/e#g:!\(g:!\(g:!\(h:codeEditor,i:\(fontScale:14;fontUsePx:'0',j:1,lang:rust,selection:](https://godbolt.org/e#g:!(g:!(g:!(h:codeEditor,i:(fontScale:14;fontUsePx:'0',j:1,lang:rust,selection:)

(endColumn:2,endLineNumber:15,positionColumn:2,positionLineNumber:15,selectionStartColumn:2,selectionStartLineNumber:15,startColumn:2,startLineNumber:15),source:pub+fn+safe_fib(n:+u32)+-

%3E+Option%3Cu64%3E+%7B%0A++++if+n+%3D%3D+0+%7B+return+Some(0)%3B+%7D%0A++++if+n+%3D%3D+1+%7B+return+Some(1)%3B+%7D%0A%0A++++let+mut+a:+u64+%3D+0;%7D%0A++++let+mut+b:+u64+%3D+1;%7D%0A%0A++++for+_+in+2..%3Dn+%7B%0A++++++//+If+addition+overflows,+return+None%0A++++++let+next+%3D+a.checked_add(b)?%3F%3B%0A++++++a+%3D+b;%7D%0A++++b+%3D+next;%7D%0A++++Some(b)%0A%7D',l:

'5',n:'0',o:'Rust+source+%231',t:'0')),k:50,l:'4',m:100,n:'0',o:',s:0,t:'0'),(g:!(h:compiler,i:(compiler:r1930,filters:

(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14;fontUsePx:'0',j:1,lang:rust,libs:!(,options:',overrides:!(,selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:

1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:+'rustc+1.93.0+

(Editor+%231',t:'0')),k:50,l:'4',n:'0',o:',s:0,t:'0')),l:'2',m:100,n:'0',o:',t:'0')),version:4

rustc 1.93.0 (Editor #1) 

rustc 1.93.0

  Compiler options... ▾

A ▾  Output... ▾  Filter... ▾  Libraries  Overrides + Add new... ▾  Add tool... ▾

```
1 <core::ops::range::RangeInclusive<T> as core::iter::range::RangeInclusiveIterat
2     sub     rsp, 72
3     mov     qword ptr [rsp], rdi
4     mov     qword ptr [rsp + 24], rdi
5     mov     rcx, rcx
6     jne     .LBB0_2
7     mov     rax, rcx
8     mov     qword ptr [rsp + 40], rax
9     mov     qword ptr [rsp + 48], rax
10    mov     qword ptr [rcx + 40], rax
11    mov     qword ptr [rcx + 48], rax
12    mov     rcx, rcx
13    cmp     eax, dword ptr [rcx + 4]
14    setbe   al
15    xor     al, -1
16    jne     .LBB0_4
17    jne     .LBB0_4
18    jne     .LBB0_4
19    .LBB0_2:
20    mov     rcx, rcx
21    mov     qword ptr [rsp], rdi
22    mov     rcx, rcx
23    mov     qword ptr [rsp + 56], rax
24    mov     qword ptr [rsp + 64], rax
25    mov     rcx, rcx
26    mov     qword ptr [rcx + 40], rax
27    mov     qword ptr [rcx + 48], rax
28    cmp     eax, dword ptr [rcx + 4]
29    setb    al
30    mov     cl, al
31    and     cl, 1
32    mov     byte ptr [rsp + 35], cl
33    test    al, 1
34    jne     .LBB0_6
35    jmp     .LBB0_5
36 .LBB0_4:
37    mov     dword ptr [rsp + 12], 0
```


KOTLIN: THE INDUSTRIAL-GRADE LANGUAGE

CONCISENESS & READABILITY (Less Boilerplate, More Power)



OLD JAVA CODE

Streamlined syntax. Reduces verbosity.

INTEROPERABILITY (Seamless Java Integration)



100%
COMPATIBILITY



Works with existing Java libraries & frameworks.

KOTLIN CORE ENGINE

NULL SAFETY (Preventing the Billion-Dollar Mistake)



Eliminates NullPointerExceptions at compile time.

NULL
LEAK

MULTIPLATFORM (Code Sharing Across Platforms)



ANDROID



iOS



WEB



DESKTOP

Share logic between Mobile, Web, & Native.

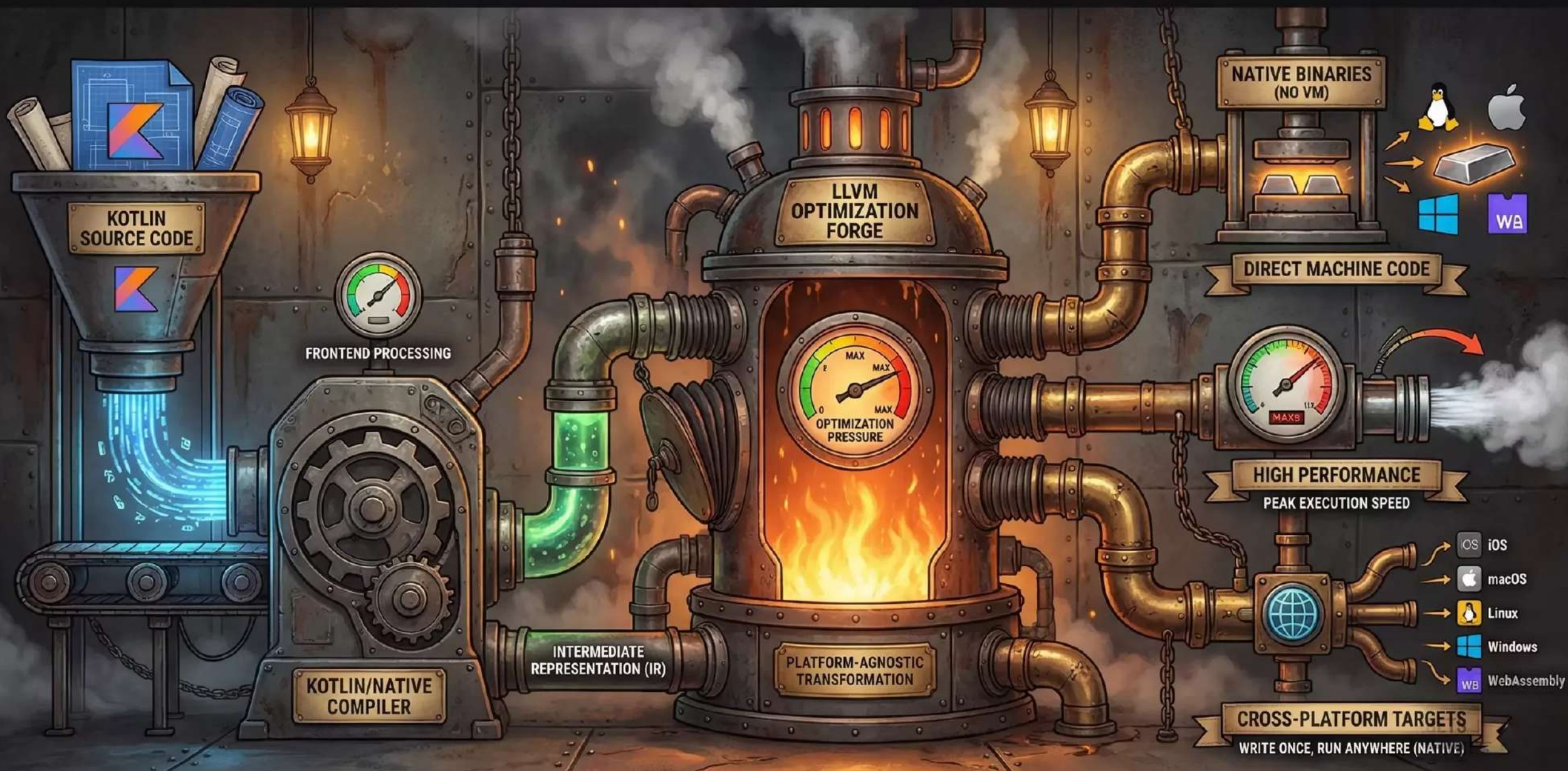
COROUTINES (Asynchronous Programming)



Simplify concurrent code. Non-blocking operations.

MODERN. PRAGMATIC. SAFE. THE ENGINE OF FUTURE DEVELOPMENT.

KOTLIN & LLVM: FORGING NATIVE POWER



Kotlin/Native leverages the LLVM toolchain to compile directly to standalone native binaries, unlocking high performance and broad platform reach without a virtual machine

Kotlin source #1

A ▾ Save/Load + Add new... ▾ Vim

Kotlin

```
1 fun fibIterative(n: Int): Long {
2     if (n <= 0) return 0
3     if (n == 1) return 1
4
5     var a = 0
6     var b = 1
7
8     // Loop from 2 up to n (inclusive)
9     for (i in 2..n) {
10        val next = a + b
11        a = b
12        b = next
13    }
14
15    return b
16 }
```

[<https://godbolt.org/z/aWK1d18ba>](https://godbolt.org/e#g:!(g:!(g:!(h:codeEditor,i:(fontScale:14;fontUsePx:'0';j:1,lang:kotlin,selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:'fun+fibIterative(n:+Int):+Long+%7B%0A++++if+(n+%3C%3D+0)+return+0%0A++++if+(n+%3D%3D+1)+return+1%0A%0A++++var+a+%3D+0L+//+Long+literal%0A++++var+b+%3D+1L%0A++++%0A++++//+Loop+from+2+up+to+n+(inclusive)%0A++++for+(i+in+2..n)+%7B%0A++++++val+next+%3D+a+%2B+b%0A++++++a+%3D+b%0A++++++b+%3D+next%0A+++++%7D%0A%0A++++return+b%0A%7D')',l:'5',n:'0',o:'Kotlin+source+%231',t:'0')',k:50,l:'4',m:100,n:'0',o:',s:0,t:'0'),(g:!(h:compiler,i:(compiler:kotlinc2220,filters:(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14;fontUsePx:'0';j:1,lang:kotlin,libs:!(,options:',overrides:!(,selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'+kotlinc+2.2.20+(Editor+%231)',t:'0')',k:50,l:'4',n:'0',o:',s:0,t:'0')',l:'2',m:100,n:'0',o:',t:'0'))',version:4
</p>
</div>
<div data-bbox=)

kotlinc 2.2.20 (Editor #1)

kotlinc 2.2.20

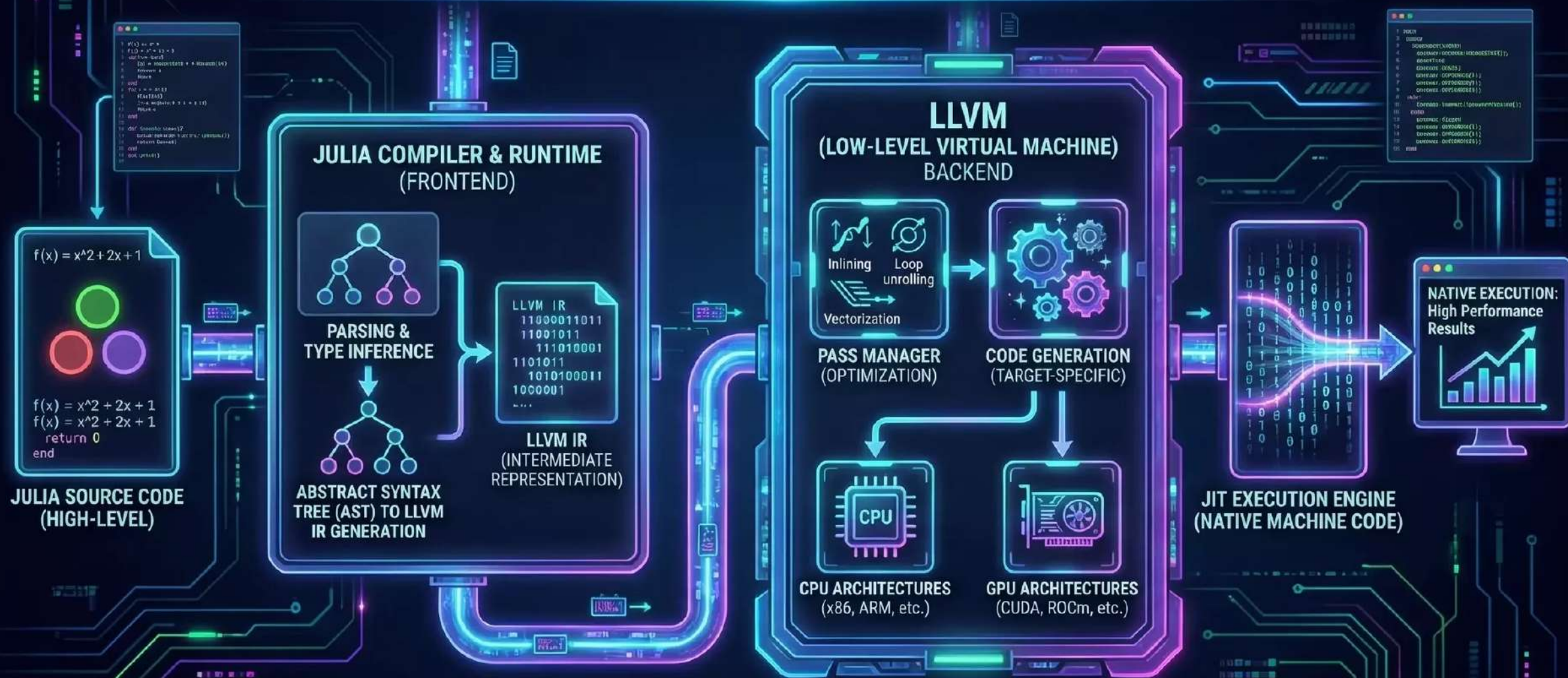
Compiler options...

A ▾ Output... ▾ Filter... ▾ Libraries ▾ Overrides + Add new... ▾ Add tool... ▾

```
1 public final class ExampleKt {
2     public static final long fibIterative(int);
3
4     1: ifgt          6
5     5: lreturn
6     7: iconst_1
7     7: lconst_1    13
8     11: lconst_0
9     12: lconst_0
10    13: lconst_0
11    14: lconst_1
12    15: lconst_1
13    16: lconst_3
14    17: lconst_5
15    18: lconst_5
16    22: iload_0
17    23: iload_0
18    26: lload_1
19    27: lload_1
20    28: ladd
21    29: lstore       6
22    31: lload_3
23    32: lstore_1
24    33: lload_1
25    34: lload_1
26    35: lstore_3
27    36: lload_3
28    38: iload_0
29    39: if_icmpeq    48
30    42: iinc        5, 1
31    45: goto        26
32    48: lload_3
33    49: lreturn
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

LocalVariableTable:

LLVM AND JULIA: DYNAMIC JIT COMPILATION



Julia leverages LLVM for dynamic, optimized JIT compilation, bridging high-level abstraction with native performance.

JULIA: HIGH-PERFORMANCE DYNAMIC PROGRAMMING

JUST-IN-TIME COMPILATION (LLVM)



NEAR-NATIVE SPEED.
COMPILS TO MACHINE CODE.

MULTIPLE DISPATCH



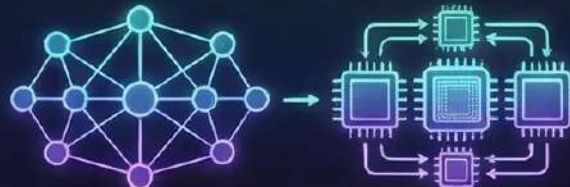
GENERIC & EXPRESSIVE.
EFFICIENT FUNCTION SELECTION.

DYNAMIC TYPING



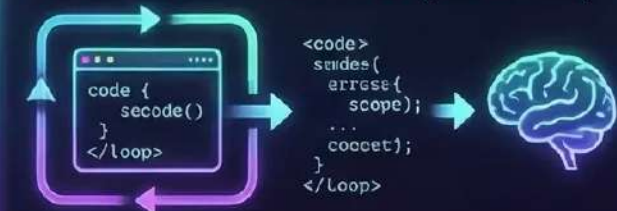
FLEXIBLE & INTERACTIVE.
RAPID PROTOTYPING.

PARALLEL & DISTRIBUTED COMPUTING



SCALABLE PERFORMANCE.
EASY CONCURRENCY.

METAPROGRAMMING (MACROS)



CODE THAT WRITES CODE.
POWERFUL ABSTRACTIONS.



BUILT-FOR-MATH & DATA SCIENCE

EFFICIENT NUMERICAL LIBRARIES.
FIRST-CLASS ARRAY SUPPORT.



Julia source #1 A ▾  Save/Load + Add new... ▾ Vim Julia

```
1 function fib_huge(n::Int)
2     if n <= 0 return BigInt(0) end
3
4     a, b = BigInt(0), BigInt(1)
5
6     for i in 1:n
7         a, b = b, a+b
8     end
9     return b
10 end
11 println(fib_huge(200))
12
13 precompile(fib_huge, {Int})
14
15
```

[\(endColumn:29,endLineNumber:14,positionColumn:29,positionLineNumber:14,selectionStartColumn:29,selectionStartLineNumber:14,startColumn:29,startLineNumber:14\),source:'function+fib_huge\(n::Int\)%0A++++if+n+%3C%3D+0+return+BigInt\(0\)+end%0A++++a,b=BigInt\(0\),BigInt\(1\)%0A++++for+i+in+2:n%0A++++a,b+=a,b+%3D+b,+a+%2B+b%0A++++end%0A++++return+b%0Aend%0A%0Aprintln\(fib_huge\(200\)\)%0Aprecompile\(fib_huge,\(Int,\)\)%0A',l:'5',n:'0',o:'Julia+source+%231',t:'0'\)\),k:50,l:'4',m:100,n:'0',o:',s:0,t:'0'\),\(g:!\(h:compiler,i:](https://godbolt.org/e#g:!(g:!(g:!(h:codeEditor,i:(fontScale:14;fontUsePx:'0',j:1,lang:julia,selection:</p></div><div data-bbox=)

(compiler:julia_111_2,filters:(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14;fontUsePx:'0',j:1,lang:julia,libs:!(,options:",overrides:!(,selection:

(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'+'Julia+1.11.2+(Editor+%231',t:'0')),k:50,l:'4',n:'0',o:',s:0,t:'0')),l:'2',m:100,n:'0',o:',t:'0')),version:4

<https://godbolt.org/z/rYs8rWoEz>

Julia 1.11.2 (Editor #1) 

Julia 1.11.2

  Compiler options...A ▾  Output... ▾  Filter... ▾  Libraries  Overrides + Add new... ▾  Add tool... ▾

```
1 julia_fib_huge_1069:                                # @julia_fib_huge_1069
2     push     rbp
3     mov     rbp, rsp
4     push    r15
5     push    r12
6     sub     rsp, 80
7     mov     rcx, 0
8     mov     rax, 0
9     mov     rdi, rcx
10    mov     rsi, rcx
11    mov     rbp, rcx
12    mov     rbp, rcx
13    mov     rbp, rcx
14    mov     rbp, rcx
15    mov     rbp, rcx
16    mov     rax, [rbp - 80]
17    mov     qword ptr [r13], rax
18    mov     rbp, rcx
19    xor     edi, edi
20    call     r15
21    mov     r12, rax
22    mov     qword ptr [rbp - 64], rax
23    mov     esi, 1
24    mov     rdi, rax
25    call     r14
26    cmp     rbx, rbx
27    jle     .LBB0_1
28    xor     edi, edi
29    call     r15
30    mov     r15, rax
31    mov     qword ptr [rbp - 64], rax
32    mov     esi, 1
33    mov     rdi, rax
34    call     r14
35    cmp     rbx, 1
```


LLVM AND SWIFT: EXTENDING & ENABLING

SWIFT SOURCE (FRONTEND)



```
func greet(name: String) {  
    print("Hello, \(name)!")  
}
```

SIL GENERATION &
HIGH-LEVEL OPTIMIZATION



SWIFT
INTERMEDIATE
LANGUAGE (SIL)

LLVM CORE (MIDDLE-END)

01010010010
10001011001
01010010100
01001001010
10001001010
01011001001

LLVM IR
GENERATION



LLVM OPTIMIZER
(PASS MANAGER)

GENERAL-PURPOSE OPTIMIZATIONS
(INLINING, VECTORIZATION, ETC.)

LLVM BACKEND (TARGET GENERATION - APP STORE)



iOS
(ARM64)



macOS
(Apple Silicon)



watchOS



tvOS



App Store

LLVM BACKEND (TARGET GENERATION - CROSS-PLATFORM)



LINUX
(x86-64, ARM64)



WINDOWS
(x86-64, ARM64)



ANDROID
(ARM64)

LLVM BACKEND (TARGET GENERATION - WEB)



WebAssembly
(WASM)



KEY ENABLER: LLVM PROVIDES CROSS-PLATFORM REACH & ROBUST OPTIMIZATION FOR SWIFT.

Swift source #1

A ▾ Save/Load + Add new... ▾ Vim

Swift

```
1 func fibFunctional(_ n: Int) -> UInt64 {
2     guard n > 1 else { return UInt64(n) }
3
4     // We start with (0, 1) and accumulate n-1 times
5     return (2...n).reduce((0, 1)) { (pair, _) in
6         (pair.1, pair.0 + pair.1)
7     }.1
8 }
```

https://godbolt.org/e#g:!(g:!(g:!(h:codeEditor,i:(fontScale:14;fontUsePx:'0',j:1,lang:swift,selection:

(endColumn:2,endLineNumber:8,positionColumn:2,positionLineNumber:8,selectionStartColumn:2,selectionStartLineNumber:8,startColumn:2,startLineNumber:8),source:'func fibFunctional(_ n: Int) -> UInt64 {

%3E+UInt64+%7B%0A++++guard+n+%3E+1+else+%7B+return+UInt64(n)+%7D%0A++++%0A++++//+We+start+with+(0,+1)+and+accumulate+n-1+times%0A++++return+(2...n).reduce((0,+1))+%7B+(pair,+_)in%0A+++++++(pair.1,+pair.0+%2B+pair.1)%0A++++%7D.1%0A%7D',l:'5',n:'0',o:'Swift+source+%231',t:'0'),k:50,l:'4',m:100,n:'0',o:',s:0,t:'0'),

(g:!(h:compiler,i:(compiler:swift62,filters:

(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14;fontUsePx:'0',j:1,lang:swift,libs:!(,options:',overrides:!

()),selection:

(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'x86-64+swiftc+6.2+(Editor+%231',t:'0'),k:50,l:'4',n:'0',o:',s:0,t:'0'),l:'2',m:100,n:'0',o:',t:'0'),version:4

x86-64 swiftc 6.2 (Editor #1) X

x86-64 swiftc 6.2

Compiler options...

A ▾ Output... ▾ Filter... ▾ Libraries ▾ Overrides + Add new... ▾ Add tool... ▾

```
1 main:
2     push    rbp
3     mov     rbp, rsp
4     xor     eax, eax
5     ret
6
7 output.fibFunctional(Swift.Int) -> Swift.UInt64:
8     push    rbp
9     mov     rbp, rsp
10    mov     rbp, rbp
11    push    rbp
12    mov     rbp, rbp
13    sub     rsp, 176
14    mov     qword ptr [rbp - 24], 0
15    mov     qword ptr [rbp - 88], rax
16    mov     qword ptr [rbp - 40], rax
17    mov     qword ptr [rbp - 40], rax
18    mov     eax, 1
19    cmp     rax, rax
20    jge     .LBB1_5
21    mov     rax, qword ptr [rbp - 96]
22    lea     rcx, [rip + 40]
23    mov     rcx, [rip + 40]
24    add     rcx, 8
25    mov     qword ptr [rbp - 104], rcx
26    mov     rcx, [rbp - 104]
27    cmp     rcx, 2
28    jne     .LBB1_3
29    xor     al, -1
30    test    al, 1
31    jne     .LBB1_3
32    lea     rdi, [rip + ".L.str.11.Fatal error"]
33    mov     esi, 11
34    mov     r9d, 2
35    lea     rcx, [rip + ".L.str.39.Range requires lowerBound <= upperBound"]
36    mov     r8d, 39
37    lea     rax, [rip + ".L.str.23.Swift/ClosedRange.swift"]
38    mov     edx, r9d
```


ZIG AND LLVM

STAGE 1: PARSE

STAGE 1: PARSE

ZIG FRONTEND
(SOURCE CODE & COMPTIME)



.zig



COMPILE-TIME
EXECUTION



STAGE 2: OPTIMIZE

LLVM MIDDLE-END
(INTERMEDIATE REPRESENTATION & OPTIMIZATION)

ZIG IR GEN



LLVM IR
(COMMON BITCODE)



LLVM OPTIMIZER
(PASS MANAGER)



STAGE 2: OPTIMIZE



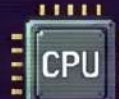
**ZIG BUILD SYSTEM
ORCHESTRATION**



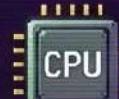
KEY SYNERGY: ZIG BUNDLES LLVM!
NO SEPARATE INSTALL NEEDED.
SEAMLESS CROSS-COMPIRATION
OUT-OF-THE-BOX.

STAGE 3: EMIT

LLVM BACKEND
(CODE GENERATION & TARGETING)



x86-64



ARM64

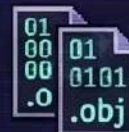


RISC-V



WASM

WASM



OBJECT FILE
EMISSION

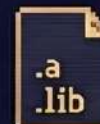
ZERO-DEPENDENCY
TOOLCHAIN

ZERO-DEPENDENCY TOOLCHAIN

FINAL ARTIFACTS



linux



.lib



.so
.dll
.dylib

HIGHLY OPTIMIZED NATIVE BINARIES
NO LIBC DEPENDENCY (OPTIONAL).

ZIG: SIMPLICITY, PERFORMANCE, AND SAFETY

SIMPLICITY



NO HIDDEN
CONTROL FLOW



MANUAL MEMORY
MANAGEMENT
(allocators)



NO
PREPROCESSOR
MAGIC

Clear, explicit code. Control is yours.



PERFORMANCE



C-COMPATIBLE
& FASTER



DIRECT LLVM
IR GENERATION



COMPTIME:
CODE AT
COMPILE-TIME

Leverages LLVM for optimized,
native binaries.

LLVM BACKEND
(CROSS-PLATFORM
POWERHOUSE)

SAFETY



OPTIONAL
TYPES (?T)



CHECKED
ARITHMETIC



COMPILE-TIME
CHECKS

Catch errors early. Robust and reliable.

> ZIG BUILD RUN --RELEASE-FAST.EXE // READY.

Zig source #1

A ▾ Save/Load + Add new... ▾ Vim

Zig

```

1  const std = @import("std");
2
3  pub fn main() !void {
4      const n: u64 = 10;
5      const result = fib_iterative(n);
6      // Print the result to console
7      std.debug.print("The {d}th Fibonacci number is: {d}\n", {}, n, result);
8
9      if (n <= 1) return n;
10     var a: u64 = 0;
11     var b: u64 = 1;
12     // Loop from 2 up to n (inclusive)
13     for (2..n+1) |_| {
14         const temp = a + b;
15         a = b;
16         b = temp;
17     }
18     return b;
19 }

```

[\(endColumn:15,endLineNumber:1,positionColumn:15,positionLineNumber:1,selectionStartColumn:15,selectionStartLineNumber:1,startColumn:15,startLineNumber:1\),source:'//+Iterative+fibonacci+sequence%0Aconst+std+%3D+@import\(%22std%22\)%3B%0A%0Apub+fn+main\(\)+!!void+%7B%0A++++const+n:+u64+%3D+10%3B%0A++++const+result+%3D+fib_iterative\(n\)%3B%0A%0A++++//+Print+the+result+to+console%0A++++std.debug.print\(%22The+%7Bd%7Dth+Fibonacci+number+is:+%7Bd%7D%5Cn%22,+.%7B+n,+result+%7D\)%3B%0A%7D%0A%0Afn+fib_iterative\(n:+u64\)+u64+%7B%0A++++if+\(n+%3C%3D+1\)+return+n%3B%0A%0A++++var+a:+u64+%3D+0%3B%0A++++var+b:+u64+%3D+1%3B%0A%0A++++//+Loop+from+2+up+to+n+\(inclusive\)%0A++++for+](https://godbolt.org/e#g:!(g:!(g:!(h:codeEditor,i:(fontScale:14;fontUsePx:'0',j:1,lang:zig,selection:

</div>
<div data-bbox=)

(2..n+%2B+1)+%7C+%7C+%7B%0A++++++const+temp+%3D+a+%2B+b%3B%0A++++++a+%3D+b%3B%0A++++++b+%3D+temp%3B%0A+++++%7D%0A%0A++++return+b%3B%0A%7D'),l:'5',n:'0',o:'Zig+source+%231',t:'0')),k:50,l:'4',n:'0',o:',s:0,t:'0'),(g:!(h:compiler,i:(compiler:ztrunk,filters:

(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14;fontUsePx:'0',j:1,lang:zig,libs:!(0),options:',overrides:!(0),selection:

(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'+zig+trunk+(Editor+%231)',t:'0')),k:50,l:'4',n:'0',o:',s:0,t:'0')),l:'2',m:100,n:'0',o:',t:'0')),version:4

zig trunk (Editor #1)

zig trunk

Compiler options... <https://godbolt.org/z/YhT176Whe> ▾

A ▾ Output... ▾ Filter... ▾ Libraries Overrides + Add new... ▾ Add tool... ▾

1 example.main:

```

3      mov     rbp, rsp
5      mov     qword ptr [rbp - 32], rdi
7      mov     esi, 10
9      mov     rdi, qword ptr [rbp - 32]
10     mov     qword ptr [rbp - 40], rax
11     mov     rdi, qword ptr [rbp - 8], rax
12     lea     rsi, [rbp - 8]
13     mov     qword ptr [rbp - 8], rax
14     xor     eax, eax
15     mov     rax, rsi
16     pop     rbp
17     mov     rax, rsi
18     mov     qword ptr [rbp - 40], rax
19     mov     qword ptr [rbp - 40], rax
20     mov     qword ptr [rbp - 40], rax
21     mov     qword ptr [rbp - 40], rax
22     sub     rsp, 96
23     mov     qword ptr [rbp - 32], rdi
24     mov     qword ptr [rbp - 24], rdi
25     mov     qword ptr [rbp - 16], rdi
26     mov     rax, 1
27     jbe     .LBB3_2
28     push    rbp
29     .LBB3_1:
30     mov     rax, qword ptr [rbp - 64], rax
31     mov     qword ptr [rbp - 32], 0
32     mov     qword ptr [rbp - 24], 1
33     mov     qword ptr [rbp - 16], 0
34     add     rax, 1
35     mov     qword ptr [rbp - 64], rax
36     setb    al
37     jnb     .LBB3_4

```


WebAssembly

Powered by
LLVM



UNIVERSAL COMPILER INFRASTRUCTURE:
Transforming diverse languages
into optimized, portable code.

**THE WEBASSEMBLY
UNIVERSE**

**DESKTOP
APPLICATIONS**

**BEYOND THE WEB
(Universal Runtime)**

LLVM empowers WebAssembly
to become the universal binary
format for all platforms, driving
the future of computing

**IoT &
BLOCKCHAIN
SMART CONTRACTS**

**PLUGIN
SYSTEMS**

**SERVER & CLOUD
(Server-Side)**

Secure, portable, and efficient
runtime outside the browser.

**EDGE
COMPUTING**

**CLOUD-NATIVE
MODULES (WASI)**

**SERVERLESS
FUNCTIONS**

MICROSERVICES

**WEB BROWSER EXPANSION
(Client-Side)**

Near-native speed in the browser,
breaking JS limits!

**GAMING & GRAPHICS
(WebGL/WebGPU)**

**HIGH-PERFORMANCE
APPS**

**AUDIO & VIDEO
PROCESSING**

**MACHINE LEARNING
INFERENCE**

**C/C++
(Clang)**

RUST

GO

**PYTHON
(via Pyodide)**

SWIFT

KOTLIN

**SOURCE CODE
INPUT**

THE LLVM TOOLCHAIN CORE

**LLVM IR
(Intermediate
Representation)**

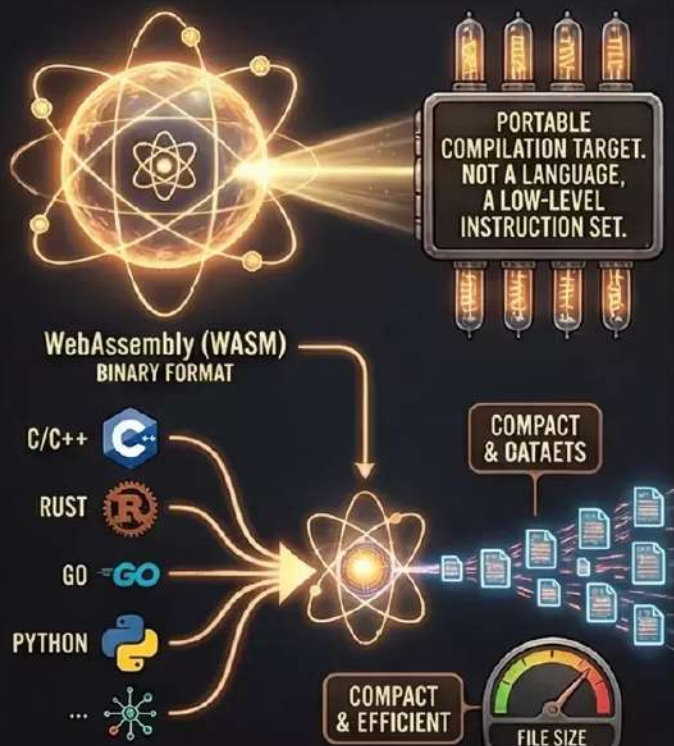
**OPTIMIZATION
PIPELINE**

**TARGET INDEPENDENT
CODE GEN**

**CROSS-PLATFORM
COMPILE**

WebAssembly: More than JavaScript

1. THE ATOMIC CORE: WHAT IS WASM?



2. POWERING THE ADVANTAGE OVER JS

DYNAMIC, JIT-COMPILED, HIGH-LEVEL. GREAT FOR INTERACTIVITY, BUT OVERHEAD.



JAVASCRIPT (JS)



NEAR-NATIVE SPEED, PREDICTABLE, SAFE. RUNS ALONGSIDE JS, NOT A REPLACEMENT.



WASM EFFICIENCY

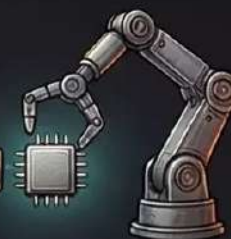


FASTER EXECUTION & SMALLER FOOTPRINT

3. BEYOND THE BROWSER HORIZON: UNIVERSAL SCOPE



SERVER-SIDE & CLOUD (SERVERLESS, CONTAINERS)



IoT & EDGE DEVICES (EMBEDDED SYSTEMS, SENSORS)



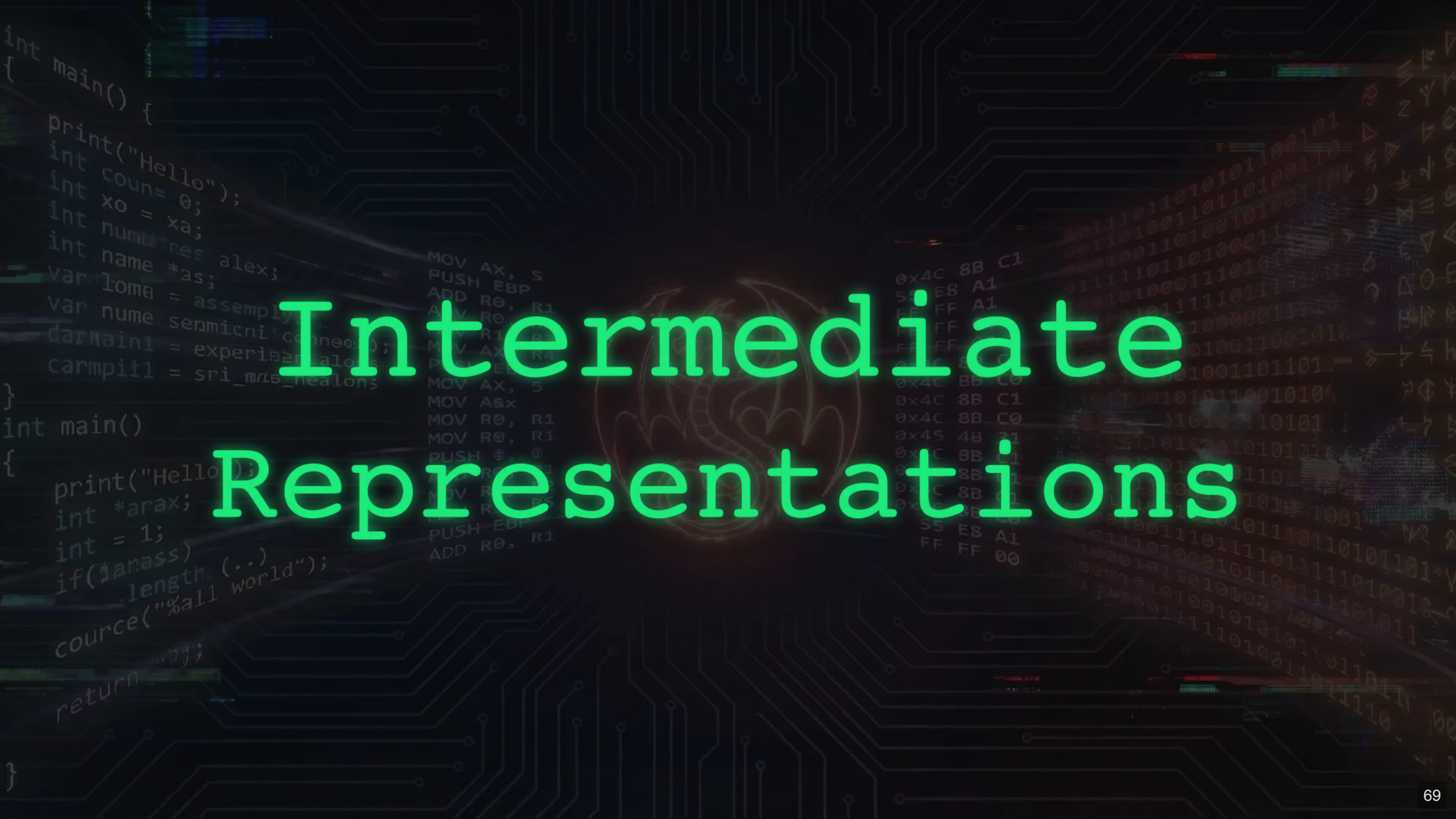
STANDALONE APPS & PLUGINS (DESKTOP, CLI TOOLS)



BLOCKCHAIN & SMART CONTRACTS

A UNIVERSAL BINARY FOR ANY PLATFORM. THE FUTURE IS PORTABLE.

Intermediate Representations



INTERMEDIATE REPRESENTATIONS

Abstracting Source Code from CPU-Specific Machine Code

LLVM IR (Low-Level Virtual Machine)



Source Code

Clang /
Frontend

LLVM IR

```
%1 = alloca i32
store i32 5, i32* %1
%2 = load i32, i32* %1
%2 = load i32, i32* %1
ret i32 %2
```

Low-Level, SSA-based,
Strongly Typed

HARDWARE ABSTRACTION LAYER (Optimizer & Backend)

CPU-Specific Machine Code
(x86, ARM, RISC-V)

.NET CIL (Common Intermediate Language)



Source Code

Roslyn
Compiler

.NET CIL

```
ldc.i4.5
stloc.0
ldloc.0
call void [mscorlib]System.Console::WriteLine(int32)
ret
```

Stack-based, Object-Oriented,
Metadata-rich

RUNTIME ABSTRACTION (CLR & JIT)

Native Machine Code
(x64, ARM64)

EXECUTABLE REALITY
(Silicon)

Java Bytecode



Source Code

javac
Compiler

Java Bytecode

```
iconst_5
istore_1
iload_1
invokestatic #2 // Method java/lang/System.out
return
```

Stack-based, JVM-targeted,
Platform-Independent

VIRTUAL MACHINE ABSTRACTION (JVM & JIT/Interpreter)

Host Machine Code
(x86, ARM, etc.)

LLVM IR, .NET CIL, & Java ByteCode:

A Nanopunk Comparison of Intermediate Architectures

LLVM IR (Low-Level Virtual Machine)



NATURE:
LANGUAGE-AGNOSTIC,
LOW-LEVEL, SSA-BASED
(STATIC SINGLE
ASSIGNMENT)

KEY FEATURES:
MODULAR OPTIMIZATION,
TARGET-INDEPENDENT,
ABSTRACTION OF
HARDWARE,
RICH TYPE SYSTEM

```
define i32 @main() {  
  %1 = alloca i32  
  ...  
}
```

LLVM IR

OPTIMIZER

OPTIMIZER

MACHINE
CODE
(JIT/AOT)

C, C++,
Rust

CLANG/LLVM
FRONTEND

.NET CIL (Common Intermediate Language)



NATURE:
STACK-BASED,
HIGH-LEVEL, TYPE-SAFE
(GC)

KEY FEATURES:
PLATFORM
INDEPENDENCE (VIA CLR),
JIT COMPILATION,
RICH METADATA,
GARBAGE COLLECTION,
INTEROPERABILITY

```
ldarg.0  
ldc.i4.s 10  
add  
stloc.0
```

.NET CIL

SECURITY/GC

JIT COMPILER
(CLR)

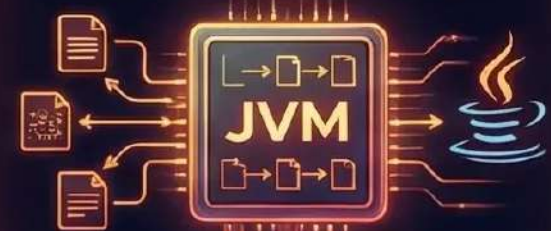
NATIVE
MACHINE
CODE

C#, F#,
VB.NET

ROSLYN
COMPILER

JIT

Java ByteCode (JVM Instruction Set)



NATURE:
STACK-BASED,
PLATFORM-INDEPENDENT
(VIA JVM)

KEY FEATURES:
WRITE ONCE
RUN ANYWHERE,
DYNAMIC LINKING,
SECURITY SANDBOX,
AUTOMATIC MEMORY
MANAGEMENT (GC)

```
iload_1  
iconst_1  
iadd  
istore_1
```

JAVA BYTECODE
(.class)

CROSS-
PLATFORM

JVM
(INTERPRETER/JIT)

HOST
MACHINE
CODE

Java,
Kotlin,
Scala

JAVAC
COMPILER

SHARED GOAL: UNIVERSAL DIGITAL BLUEPRINTS FOR EFFICIENT, SECURE, AND PORTABLE EXECUTION IN THE NANOTECH AGE.

ASM, IR, & MACHINECODE

ASM (ASSEMBLY LANGUAGE)

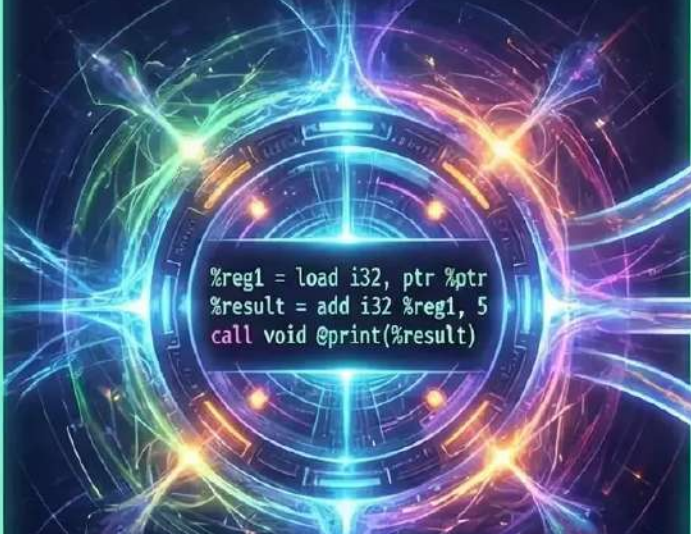


```
MOV EAX, 0xFF
ADD EBX, 10
JMP LOOP_START

.....
MOV EAX, 0xFF_ptr_START
ADD EBX, 10
ADD EBX, 10
JMP LOOP_START
```

- HUMAN-READABLE INSTRUCTIONS
- CPU-SPECIFIC MNEMONICS
- LOW-LEVEL ABSTRACTION
- DIRECT HARDWARE CONTROL

IR (INTERMEDIATE REPRESENTATION)



```
%reg1 = load i32, ptr %ptr
%result = add i32 %reg1, 5
call void @print(%result)
```

- PLATFORM-AGNOSTIC HUB
- OPTIMIZATION-READY FORM
- COMPILER BRIDGE
- TARGET-INDEPENDENT CODE

MACHINECODE (BINARY EXECUTION)



```
10110001
10110001
01001110
11001010
00011111
01110001
01001110
11001010
00011111
01110001
11001110
11001010
10011111
```

- EXECUTABLE BINARY
- CPU-DIRECT INSTRUCTIONS
- BINARY FORMAT
- NATIVE EXECUTION

THE EVOLUTION OF CODE: FROM HUMAN-READABLE TO MACHINE-EXECUTABLE, BRIDGED BY NANOTECH OPTIMIZATION.

MOV EAX, 0xFF
ADD EBX, 10
JMP LOOP_START
.....
MOV EAX, 0xFF_ptr_START
ADD EBX, 10
ADD EBX, 10
JMP LOOP_START

00000000
J 00000000
00000001
P 00000001
00000002
J 00000002
00000003
P 00000003
00000004
J 00000004
00000005
P 00000005
00000006
J 00000006
00000007
P 00000007
00000008
J 00000008
00000009
P 00000009
0000000A
J 0000000A
0000000B
P 0000000B
0000000C
J 0000000C
0000000D
P 0000000D
0000000E
J 0000000E
0000000F
P 0000000F
00000010
J 00000010
00000011
P 00000011
00000012
J 00000012
00000013
P 00000013
00000014
J 00000014
00000015
P 00000015
00000016
J 00000016
00000017
P 00000017
00000018
J 00000018
00000019
P 00000019
0000001A
J 0000001A
0000001B
P 0000001B
0000001C
J 0000001C
0000001D
P 0000001D
0000001E
J 0000001E
0000001F
P 0000001F
00000020
J 00000020
00000021
P 00000021
00000022
J 00000022
00000023
P 00000023
00000024
J 00000024
00000025
P 00000025
00000026
J 00000026
00000027
P 00000027
00000028
J 00000028
00000029
P 00000029
0000002A
J 0000002A
0000002B
P 0000002B
0000002C
J 0000002C
0000002D
P 0000002D
0000002E
J 0000002E
0000002F
P 0000002F
00000030
J 00000030
00000031
P 00000031
00000032
J 00000032
00000033
P 00000033
00000034
J 00000034
00000035
P 00000035
00000036
J 00000036
00000037
P 00000037
00000038
J 00000038
00000039
P 00000039
0000003A
J 0000003A
0000003B
P 0000003B
0000003C
J 0000003C
0000003D
P 0000003D
0000003E
J 0000003E
0000003F
P 0000003F
00000040
J 00000040
00000041
P 00000041
00000042
J 00000042
00000043
P 00000043
00000044
J 00000044
00000045
P 00000045
00000046
J 00000046
00000047
P 00000047
00000048
J 00000048
00000049
P 00000049
0000004A
J 0000004A
0000004B
P 0000004B
0000004C
J 0000004C
0000004D
P 0000004D
0000004E
J 0000004E
0000004F
P 0000004F
00000050
J 00000050
00000051
P 00000051
00000052
J 00000052
00000053
P 00000053
00000054
J 00000054
00000055
P 00000055
00000056
J 00000056
00000057
P 00000057
00000058
J 00000058
00000059
P 00000059
0000005A
J 0000005A
0000005B
P 0000005B
0000005C
J 0000005C
0000005D
P 0000005D
0000005E
J 0000005E
0000005F
P 0000005F
00000060
J 00000060
00000061
P 00000061
00000062
J 00000062
00000063
P 00000063
00000064
J 00000064
00000065
P 00000065
00000066
J 00000066
00000067
P 00000067
00000068
J 00000068
00000069
P 00000069
0000006A
J 0000006A
0000006B
P 0000006B
0000006C
J 0000006C
0000006D
P 0000006D
0000006E
J 0000006E
0000006F
P 0000006F
00000070
J 00000070
00000071
P 00000071
00000072
J 00000072
00000073
P 00000073
00000074
J 00000074
00000075
P 00000075
00000076
J 00000076
00000077
P 00000077
00000078
J 00000078
00000079
P 00000079
0000007A
J 0000007A
0000007B
P 0000007B
0000007C
J 0000007C
0000007D
P 0000007D
0000007E
J 0000007E
0000007F
P 0000007F
00000080
J 00000080
00000081
P 00000081
00000082
J 00000082
00000083
P 00000083
00000084
J 00000084
00000085
P 00000085
00000086
J 00000086
00000087
P 00000087
00000088
J 00000088
00000089
P 00000089
0000008A
J 0000008A
0000008B
P 0000008B
0000008C
J 0000008C
0000008D
P 0000008D
0000008E
J 0000008E
0000008F
P 0000008F
00000090
J 00000090
00000091
P 00000091
00000092
J 00000092
00000093
P 00000093
00000094
J 00000094
00000095
P 00000095
00000096
J 00000096
00000097
P 00000097
00000098
J 00000098
00000099
P 00000099
0000009A
J 0000009A
0000009B
P 0000009B
0000009C
J 0000009C
0000009D
P 0000009D
0000009E
J 0000009E
0000009F
P 0000009F
000000A0
J 000000A0
000000A1
P 000000A1
000000A2
J 000000A2
000000A3
P 000000A3
000000A4
J 000000A4
000000A5
P 000000A5
000000A6
J 000000A6
000000A7
P 000000A7
000000A8
J 000000A8
000000A9
P 000000A9
000000AA
J 000000AA
000000AB
P 000000AB
000000AC
J 000000AC
000000AD
P 000000AD
000000AE
J 000000AE
000000AF
P 000000AF
000000B0
J 000000B0
000000B1
P 000000B1
000000B2
J 000000B



```

1  MOVEA     A0, A1
2  LDR       D0, #0
3  LDR       D1, #1
4  LDR       D2, #2
5  LDR       D3, #3
6  LDR       D4, #4
7  LDR       D5, #5
8  LDR       D6, #6
9  LDR       D7, #7
10 LDR       D8, #8
11 LDR       D9, #9
12 LDR       D10, #10
13 LDR       D11, #11
14 LDR       D12, #12
15 LDR       D13, #13
16 LDR       D14, #14
17 LDR       D15, #15
18 LDR       D16, #16
19 LDR       D17, #17
20 LDR       D18, #18
21 LDR       D19, #19
22 LDR       D20, #20
23 LDR       D21, #21
24 LDR       D22, #22
25 LDR       D23, #23
26 LDR       D24, #24
27 LDR       D25, #25
28 LDR       D26, #26
29 LDR       D27, #27
30 LDR       D28, #28
31 LDR       D29, #29
32 LDR       D30, #30
33 LDR       D31, #31
34 LDR       D32, #32
35 LDR       D33, #33
36 LDR       D34, #34
37 LDR       D35, #35
38 LDR       D36, #36
39 LDR       D37, #37
40 LDR       D38, #38
41 LDR       D39, #39
42 LDR       D40, #40
43 LDR       D41, #41
44 LDR       D42, #42
45 LDR       D43, #43
46 LDR       D44, #44
47 LDR       D45, #45
48 LDR       D46, #46
49 LDR       D47, #47
50 LDR       D48, #48
51 LDR       D49, #49
52 LDR       D50, #50
53 LDR       D51, #51
54 LDR       D52, #52
55 LDR       D53, #53
56 LDR       D54, #54
57 LDR       D55, #55
58 LDR       D56, #56
59 LDR       D57, #57
60 LDR       D58, #58
61 LDR       D59, #59
62 LDR       D60, #60
63 LDR       D61, #61
64 LDR       D62, #62
65 LDR       D63, #63
66 LDR       D64, #64
67 LDR       D65, #65
68 LDR       D66, #66
69 LDR       D67, #67
70 LDR       D68, #68
71 LDR       D69, #69
72 LDR       D70, #70
73 LDR       D71, #71
74 LDR       D72, #72
75 LDR       D73, #73
76 LDR       D74, #74
77 LDR       D75, #75
78 LDR       D76, #76
79 LDR       D77, #77
80 LDR       D78, #78
81 LDR       D79, #79
82 LDR       D80, #80
83 LDR       D81, #81
84 LDR       D82, #82
85 LDR       D83, #83
86 LDR       D84, #84
87 LDR       D85, #85
88 LDR       D86, #86
89 LDR       D87, #87
90 LDR       D88, #88
91 LDR       D89, #89
92 LDR       D90, #90
93 LDR       D91, #91
94 LDR       D92, #92
95 LDR       D93, #93
96 LDR       D94, #94
97 LDR       D95, #95
98 LDR       D96, #96
99 LDR       D97, #97
100 LDR       D98, #98
101 LDR       D99, #99
102 LDR       D100, #100
103 LDR       D101, #101
104 LDR       D102, #102
105 LDR       D103, #103
106 LDR       D104, #104
107 LDR       D105, #105
108 LDR       D106, #106
109 LDR       D107, #107
110 LDR       D108, #108
111 LDR       D109, #109
112 LDR       D110, #110
113 LDR       D111, #111
114 LDR       D112, #112
115 LDR       D113, #113
116 LDR       D114, #114
117 LDR       D115, #115
118 LDR       D116, #116
119 LDR       D117, #117
120 LDR       D118, #118
121 LDR       D119, #119
122 LDR       D120, #120
123 LDR       D121, #121
124 LDR       D122, #122
125 LDR       D123, #123
126 LDR       D124, #124
127 LDR       D125, #125
128 LDR       D126, #126
129 LDR       D127, #127
130 LDR       D128, #128
131 LDR       D129, #129
132 LDR       D130, #130
133 LDR       D131, #131
134 LDR       D132, #132
135 LDR       D133, #133
136 LDR       D134, #134
137 LDR       D135, #135
138 LDR       D136, #136
139 LDR       D137, #137
140 LDR       D138, #138
141 LDR       D139, #139
142 LDR       D140, #140
143 LDR       D141, #141
144 LDR       D142, #142
145 LDR       D143, #143
146 LDR       D144, #144
147 LDR       D145, #145
148 LDR       D146, #146
149 LDR       D147, #147
150 LDR       D148, #148
151 LDR       D149, #149
152 LDR       D150, #150
153 LDR       D151, #151
154 LDR       D152, #152
155 LDR       D153, #153
156 LDR       D154, #154
157 LDR       D155, #155
158 LDR       D156, #156
159 LDR       D157, #157
160 LDR       D158, #158
161 LDR       D159, #159
162 LDR       D160, #160
163 LDR       D161, #161
164 LDR       D162, #162
165 LDR       D163, #163
166 LDR       D164, #164
167 LDR       D165, #165
168 LDR       D166, #166
169 LDR       D167, #167
170 LDR       D168, #168
171 LDR       D169, #169
172 LDR       D170, #170
173 LDR       D171, #171
174 LDR       D172, #172
175 LDR       D173, #173
176 LDR       D174, #174
177 LDR       D175, #175
178 LDR       D176, #176
179 LDR       D177, #177
180 LDR       D178, #178
181 LDR       D179, #179
182 LDR       D180, #180
183 LDR       D181, #181
184 LDR       D182, #182
185 LDR       D183, #183
186 LDR       D184, #184
187 LDR       D185, #185
188 LDR       D186, #186
189 LDR       D187, #187
190 LDR       D188, #188
191 LDR       D189, #189
192 LDR       D190, #190
193 LDR       D191, #191
194 LDR       D192, #192
195 LDR       D193, #193
196 LDR       D194, #194
197 LDR       D195, #195
198 LDR       D196, #196
199 LDR       D197, #197
200 LDR       D198, #198
201 LDR       D199, #199
202 LDR       D200, #200
203 LDR       D201, #201
204 LDR       D202, #202
205 LDR       D203, #203
206 LDR       D204, #204
207 LDR       D205, #205
208 LDR       D206, #206
209 LDR       D207, #207
210 LDR       D208, #208
211 LDR       D209, #209
212 LDR       D210, #210
213 LDR       D211, #211
214 LDR       D212, #212
215 LDR       D213, #213
216 LDR       D214, #214
217 LDR       D215, #215
218 LDR       D216, #216
219 LDR       D217, #217
220 LDR       D218, #218
221 LDR       D219, #219
222 LDR       D220, #220
223 LDR       D221, #221
224 LDR       D222, #222
225 LDR       D223, #223
226 LDR       D224, #224
227 LDR       D225, #225
228 LDR       D226, #226
229 LDR       D227, #227
230 LDR       D228, #228
231 LDR       D229, #229
232 LDR       D230, #230
233 LDR       D231, #231
234 LDR       D232, #232
235 LDR       D233, #233
236 LDR       D234, #234
237 LDR       D235, #235
238 LDR       D236, #236
239 LDR       D237, #237
240 LDR       D238, #238
241 LDR       D239, #239
242 LDR       D240, #240
243 LDR       D241, #241
244 LDR       D242, #242
245 LDR       D243, #243
246 LDR       D244, #244
247 LDR       D245, #245
248 LDR       D246, #246
249 LDR       D247, #247
250 LDR       D248, #248
251 LDR       D249, #249
252 LDR       D250, #250
253 LDR       D251, #251
254 LDR       D252, #252
255 LDR       D253, #253
256 LDR       D254, #254
25
```



DIRECT HARDWARE CONTROL

A futuristic, glowing blue and purple circular interface, resembling a high-tech control panel or a stylized atom. The interface features concentric rings and a central display area. The background is dark, with bright blue and purple light streaks radiating outwards. The central display shows the following assembly code:

```
%reg1 = load i32, ptr %ptr  
%result = add i32 %reg1, 5  
call void @print(%result)
```



**TARGET-INDEPENDENT
CODE**



NATIVE EXECUTION

72

IEEE 754

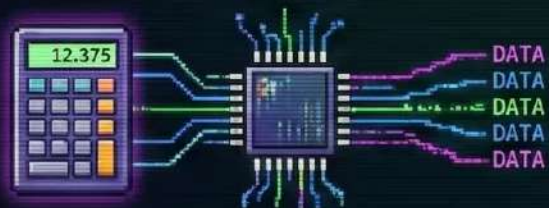
Floating Point

73

FLOATING POINT:

DECODING THE DECIMAL MATRIX & IEEE 754 STANDARD

REPRESENTATION: THE BIT SLICER



DECIMAL 12.375 → SCIENTIFIC: 1.2375×10^1

BIT BREAKDOWN



SIGN: +/-,
EXPONENT: SCALE,
MANTISSA: PRECISION.

BIAS ADJ.

ARITHMETIC & ISSUES: THE PRECISION TRAP



ALIGN EXPONENTS, ADD/SUB
MANTISSAS, NORMALIZE.
WARNING: ROUNDING ERRORS,
OVERFLOW, UNDERFLOW!

IEEE 754 STANDARD: THE UNIVERSAL PROTOCOL



DEFINES FORMATS, ROUNDING MODES,
SPECIAL VALUES, THE GLOBAL
LANGUAGE OF COMPUTATIONAL MATH.

```
>_ SYSTEM READY.  
IEEE 754 COMPLIANT.  
ACCESSING FLOATING POINT UNIT...  
[OK]
```


IEEE 754: THE FLOATING-POINT MATRIX REVEALED

32-BIT FLOAT STRUCTURE (BINARY)

+/-

SIGN (S)

1 BIT

0 = POSITIVE,
1 = NEGATIVE.
POLARITY SWITCH.



EXPONENT (E)

8 BITS

STORED AS 'BIASED'
VALUE (E - 127).
MAGNITUDE SCALER.



MANTISSA (M)

23 BITS

FRACTIONAL PART.
NORMALIZED WITH HIDDEN '1.'
PRECISION ENGINE.

DEFINES
SIGNIFICANT DIGITS.
THE DATA PAYLOAD.

FLIPS THE VALUE'S
DIRECTION IN THE
DIGITAL VOID.

SHIFTS THE
RADIX POINT.
POWER OF 2.

SPECIAL VALUES - EDGE CASES & ANOMALIES

∞

EXP=255, M=0:
INFINITY (∞).
OVERFLOW BOUNDARY.

NaN

EXP=255, M $\neq$ 0:
NaN (NOT A NUMBER).
CALCULATION ERROR.

>_ SYSTEM STATUS: DECODED. ACCESSING FLOATING-POINT UNIT. [EXECUTE]

IEEE 754

The standard for floating points

Established in 1985

lukaskollmer.de/ieee-754-visualizer/

h-schmidt.net/FloatConverter/

standards.ieee.org/ieee/754/6210/

en.wikipedia.org/wiki/IEEE_754

[/wiki/Single-precision](http://en.wikipedia.org/wiki/IEEE_754#/wiki/Single-precision)

[/wiki/Double-precision](http://en.wikipedia.org/wiki/IEEE_754#/wiki/Double-precision)

IEEE 754 Visualizer

binary16 binary32 binary64

0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 1 0 0 0 0 1 1 1 1 1 0 1 1
0 1 0 1 0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 0 1 1 0 1 0 0 0 1 1 0 0 0

3.141592653589793

$= (-1)^0 * 2^{(1024 - 1023)} * 1.5707963267948966$

IEEE 754 Converter, 2024-02

	Sign	Exponent	Mantissa
Value:	+1	2^1	$1 + 0.5707963705062866$
Encoded as:	0	128	4788187
Binary:	☐	☒ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	☒ ☐ ☐ ☒ ☐ ☐ ☒ ☐ ☐ ☐ ☒ ☒ ☒ ☒ ☒ ☐ ☒ ☒ ☐ ☒ ☒
Decimal Representation	3.1415926535897932384626433832795028841971693993751058		
Value actually stored in float:	3.1415927410125732421875		
Error due to conversion:	0.0000000874227800037248566167204971158		
Binary Representation	01000000010010010000111111011011		
Hexadecimal Representation	40490fdb		

